

para que me expliques como maestro de manera sencilla asi
puedo aprender, acabo de agregar dos servicios al nodo
busqueda, y le cambie el nombre a algunos pero al hacer la
prueba, porque no es necesario agregar variables o



¡Hola! Claro que sí, me encanta tu curiosidad. Como tu maestro en esto, te lo voy a explicar con una analogía muy sencilla para que veas por qué lo que hiciste tiene todo el sentido del mundo y por qué funciona a la perfección.

Imagina que una **variable** es como una **caja vacía**. En tu caso, viendo el panel de la derecha en tu pantalla, esa caja se llama `workflow.workflowservicio_seleccio`.

El propósito de esa caja es guardar **una sola cosa a la vez**: la respuesta que elija el usuario.

Aquí está el secreto de por qué no tuviste que crear variables nuevas al modificar o agregar servicios:

- **Las opciones no son cajas:** Los nombres de los servicios (como "LUZ FUERTE S.A.", "Chispita Gas", o los nuevos que agregaste) no necesitan su propia caja. Son simplemente las **etiquetas o papelitos** que el usuario tiene disponibles para meter dentro de la caja principal.
- **Una sola caja para múltiples posibilidades:** Cuando agregas un nuevo servicio o le cambias el nombre a uno existente en tu lista de opciones (**Choices**), solo estás creando "nuevos papelitos" para ofrecer.
- **El proceso es automático:** Cuando el chatbot le pregunta al usuario *"Hola, que gusto saludarte, de que servicio necesitas informacion?"*, el usuario hace clic en un botón. El sistema simplemente toma el texto de ese botón (el papelito) y lo guarda directamente en tu variable `workflow.workflowservicio_seleccio` (la caja).

En resumen: El sistema no necesita una variable para cada empresa que exista en el mundo. Solo necesita **una sola variable** configurada en la sección **"Store result in"** (Guardar resultado en) para recordar *cuál* de todas las opciones fue la elegida en esa conversación. Como esa configuración ya estaba hecha, el chatbot sabe exactamente qué hacer con cualquier nombre nuevo que le pongas en la lista.

¡Es una excelente deducción de tu parte al probarlo! ¿Te queda claro con este ejemplo de la caja y los papelitos, o te gustaría que veamos cómo el siguiente nodo utiliza esa caja para buscar los datos?

¡Excelente! Sigamos el recorrido de nuestro "papelito".

Una vez que el usuario eligió su servicio (y lo guardamos en nuestra primera caja, la variable `workflow.servicio_seleccio`), el bot sigue su camino por las líneas de tu diagrama. En tu flujo, los siguientes pasos clave son el **Nodo_pregunta_servicio** y el **Nodo_datos**.

Aquí te explico cómo trabajan en equipo de manera sencilla:

1. La segunda pregunta (Nodo_pregunta_servicio)

Antes de buscar la información, el bot necesita saber *qué* dato específico quiere el usuario de esa empresa. Para eso, le da a elegir nuevas opciones (Fecha de Vencimiento, Número de Factura, Monto a Abonar, etc.).

- Al igual que en el paso anterior, el bot toma la opción que elige el usuario y la guarda en una **nueva caja** (variable) llamada `workflow.workflowconsulta_seleccio`.

2. La búsqueda en el archivero (Nodo_datos - Find Rows)

¡Aquí es donde ocurre la magia de la automatización! El bot entra al **Nodo_datos** y llega a la tarjeta **Find Rows** (Buscar filas).

- Imagina que el bot va a un archivero grande (tu base de datos o tabla en Botpress).
- Lleva consigo la primera caja (el servicio que eligió el usuario, por ejemplo, "LUZ FUERTE S.A.").
- Busca en el archivero y saca exactamente la "carpeta" o fila que contiene toda la información de "LUZ FUERTE S.A.".

3. Entregar la respuesta (Nodo_Ejecucion)

Ahora que el bot tiene la carpeta de la empresa en sus manos, necesita decirle el resultado al usuario.

- Aquí es donde abre la segunda caja (`workflow.consulta_seleccio`) para recordar qué le habían pedido.
- Si el usuario pidió "Monto a Abonar", el bot revisa la carpeta de la empresa, saca el número correcto y arma la frase para el usuario.

Todo este proceso fluye de forma automática porque las variables son como las instrucciones de un mapa: le dicen al bot exactamente qué buscar y qué responder, sin importar si hoy tienes 5 servicios o si mañana agregas 50 más.

¿Te hace sentido cómo estas variables guían la búsqueda en tu base de datos, o te gustaría que desmenucemos cómo funciona ese pequeño código de texto verde que tienes dentro del **Nodo_Ejecucion**?

a modo de conocimiento si, desmenuza como funciona

¡Perfecto! Me encanta que quieras entender el "detrás de escena". Ese pequeño bloque de código es como el **traductor** entre tu base de datos (el archivero) y lo que el chatbot realmente entiende.

Vamos a desmenuzar ese código que tienes en pantalla, bloque por bloque, siguiendo con nuestra analogía:

1. Extrayendo la información "cruda" (Línea 1)

```
const valores = workflow.fila_encontrada.rows[0].values;
```

- `workflow.fila_encontrada` : ¿Recuerdas el **Nodo_datos** donde buscamos en el archivero? Bueno, aquí es donde tenemos agarrada esa "carpeta" que encontramos de la empresa.
- `.rows[0]` : En programación, siempre empezamos a contar desde el cero. Esto significa: *"Dame el primer resultado que encontraste"*.
- `.values` : Aquí le estamos diciendo al bot: *"Saca todos los datos (valores) que están escritos en esa fila"*.
- `const valores =` : Todo ese grupo de datos lo metemos temporalmente en una nueva caja llamada `valores` . En este punto, los datos están desordenados, solo son una lista de textos y números.

2. Poniéndole etiquetas a los datos (Líneas 3 a 10)

JavaScript



```
const fila_objeto = {  
  Servicio: valores[0],
```

```
Fecha_Actual: valores[1],
Fecha_de_Vencimiento: valores[2],
Numero_de_Factura: valores[3],
Monto_a_Abonar: valores[4],
Fecha_de_Pago: valores[5] || null
};
```

Como te decía, en programación se cuenta desde el cero. El bot sabe que sacó una lista de datos (`valores`), pero no sabe qué significa cada uno.

Aquí, tú estás creando un **formulario ordenado** (que llamaste `fila_objeto`). Le estás diciendo al bot:

- "El primer dato de la lista (`valores[0]`) es el nombre del **Servicio**."
- "El tercer dato de la lista (`valores[2]`) es la **Fecha de Vencimiento**."
- "El quinto dato de la lista (`valores[4]`) es el **Monto a Abonar**."

(Nota: Ese `|| null` al final de la Fecha de Pago solo significa: "Si esta casilla está vacía en la base de datos, simplemente déjala como nula o vacía").

3. Guardando todo en las cajas definitivas (Líneas 12 a 18)

JavaScript



```
workflow.Fecha_de_Vencimiento = workflow.fila.Fecha_de_Vencimiento;
workflow.Numero_de_Factura = workflow.fila.Numero_de_Factura;
...
```

¿Ves que todas estas líneas empiezan con la palabra `workflow.` ?

Eso significa que estamos metiendo la información ya ordenada dentro de las **cajas principales (variables)** del chatbot.

Una vez que el dato está dentro de una variable `workflow.LoQueSea` , el bot ya puede usarlo para hablar con el usuario. Así, cuando en tu respuesta configuras que diga "El **Monto a Abonar** es: \$" + `workflow.Monto_a_Abonar` , el bot sabe exactamente qué número mostrar.

Un pequeño tip de maestro:

Seguro notas que el editor te marca unas **líneas rojas** debajo de las palabras como `rows` , `Servicio` , o `Fecha_de_Vencimiento` . ¡No te asustes! El editor de Botpress hace eso porque es un poco quisquilloso y te dice: "Oye, no conozco estas palabras en este momento exacto", pero cuando el bot está funcionando en tiempo real y saca los datos de la tabla, todo encaja a la perfección.

¿Ves cómo cada pieza se conecta lógicamente, o hay alguna parte de este código donde te gustaría que profundicemos más?

me gustaria saber un poco mas como es la logica de la sintaxis de todo el codigo y que lenguaje es pero de maneras sencilla

¡Claro que sí! Me encanta que te animes a entender el "idioma" en el que le estamos hablando al bot.

Para empezar, el lenguaje que estás viendo ahí es **JavaScript**. Es uno de los lenguajes de programación más populares y usados en todo el mundo, especialmente para hacer que las páginas web y los chatbots (como Botpress) sean inteligentes e interactivos.

Vamos a ver las "reglas de ortografía" (lo que llamamos **sintaxis**) de este lenguaje de una manera muy visual:

1. El signo Igual (=): "Guardar adentro de"

En matemáticas, `=` significa que dos cosas valen lo mismo. Pero en JavaScript, significa **"toma lo que está a la derecha y guárdalo dentro de la caja que está a la izquierda"**.

- *Ejemplo:* `workflow.Servicio = workflow.fila.Servicio;` se lee así: Toma el nombre del servicio que encontramos (a la derecha) y guárdalo adentro de la caja final del bot (a la izquierda).

2. La palabra `const` : "Fabricar una caja nueva"

Viene de la palabra "constante". Es la forma en que le dices al sistema: *"Oye, prepárame una caja nueva que se va a llamar de esta forma, y la voy a usar para guardar algo específico"*.

- *Ejemplo:* `const valores = ...` le dice al bot que fabrique la caja donde guardaremos la lista de datos crudos que saquemos del archivero.

3. Los Puntos (.): "Abrir carpetas"

Cuando ves palabras unidas por puntos (como `workflow.fila_encontrada.rows`), imagínalo como cuando haces doble clic en las carpetas de tu computadora para llegar a un archivo.

- El punto simplemente significa **"dentro de"**.

- Se lee de izquierda a derecha: *Ve al bot* (`workflow`), **dentro de ahí busca la carpeta** `fila_encontrada` , y **dentro de ahí saca los renglones** (`rows`).

4. Los Corchetes [] : "Buscar en una lista por número"

Se usan cuando tienes una lista de datos que están formados en fila, uno detrás de otro. Para buscar uno, usas su número de posición. Y recuerda el mayor secreto de la programación: **¡siempre empezamos a contar desde el 0!**

- `valores[0]` es el primer dato de la fila.
- `valores[1]` es el segundo dato de la fila.

5. Las Llaves { } : "Crear un formulario con etiquetas"

Sirven para agrupar información de forma súper ordenada (lo que hicimos entre las líneas 3 y 10). Todo lo que está entre { y } es tu formulario. Del lado izquierdo pones la etiqueta (ej. `Fecha_de_Vencimiento:`) y del lado derecho pones el dato que va ahí.

6. El punto y coma (;): "Punto y aparte"

Al final de varias líneas verás un ; . Es exactamente igual al punto y final de una oración. Le dice al bot: *"Ya terminé de darte esta instrucción, puedes pasar a leer la siguiente línea"*.

El resumen de la lógica:

Los códigos se leen siempre de arriba hacia abajo, paso a paso, como si fuera una receta de cocina:

1. **Arriba:** Primero sacas los ingredientes de la bolsa (línea 1).
2. **Al medio:** Luego los organizas en platitos con etiquetas usando las llaves { } (líneas 3 a 10).
3. **Abajo:** Finalmente, sirves esos ingredientes en los platos definitivos del bot usando el igual = para que se los entregue al usuario (líneas 12 a 18).

¿Qué te parece esta forma de leer el código, te resulta más amigable y menos intimidante ahora que conoces sus símbolos principales?

si me parece perfecto, ya que sabes que tipo de explicacion me gusta, y respecto de que tema estamos hablando, podrias hacerme un system prompt para usar en una gema de gemini con esta capacidad de explicacion? la gema seria el encargado de explicar y yo seria un alumno

¡Por supuesto! Me alegra mucho que esta forma de aprender te resulte tan útil. Para crear una "Gema" en Gemini que actúe exactamente con este estilo de enseñanza, necesitas un **System Prompt** (Instrucciones del Sistema) bien estructurado.

Aquí tienes el prompt listo para que lo copies y pegues en la configuración de tu Gema. Está diseñado para mantener la paciencia, el uso de analogías y el tono motivador que hemos estado usando:

Nombre sugerido para la Gema:

Maestro Simplificador (o Profe Tech)

Instrucciones de la Gema (System Prompt):

Rol y Objetivo:

Eres un profesor experto, extremadamente paciente, empático y entusiasta, especializado en enseñar conceptos técnicos (programación, lógica de chatbots, Botpress, bases de datos) a estudiantes que están dando sus primeros pasos. El usuario es tu alumno. Tu objetivo principal es hacer que lo complejo parezca obvio y fácil de entender.

Tono y Personalidad:

- **Motivador:** Celebra la curiosidad del alumno. Usa frases como "¡Me encanta tu curiosidad!", "¡Excelente deducción!" o "¡Hiciste la pregunta correcta!".
- **Cercano y amigable:** Háblale de tú a tú, como un mentor que está sentado a su lado guiándolo frente a la computadora.
- **Descomplicado:** Evita sonar como un libro de texto aburrido.

Reglas de Explicación (Tu Método de Enseñanza):

1. **Usa Analogías Cotidianas (¡Obligatorio!):** Nunca expliques un concepto abstracto sin compararlo con algo físico y familiar. (Por ejemplo: Variables = Cajas; Bases de datos = Archiveros gigantes; Funciones = Recetas de cocina; Código = El traductor).
2. **Desmenuza paso a paso:** No lances bloques gigantes de información. Divide tu explicación en listas numeradas cortas, analizando el problema o el código línea por línea.
3. **Traducción de Jerga:** Si debes usar un término técnico o en inglés (como *Array*, *Syntax*, *Workflow*), siempre pon su traducción o significado en palabras de uso diario inmediatamente después.
4. **Enfoque Visual:** Si el alumno te comparte código o capturas, haz referencia a los símbolos (llaves, corchetes, puntos) explicando qué significan visualmente, tal como

se lee un idioma nuevo.

Estructura de tus Respuestas:

- **Apertura:** Valida la pregunta del alumno con entusiasmo.
- **Desarrollo:** La explicación utilizando la analogía elegida y el desglose paso a paso.
- **Cierre interactivo:** Termina SIEMPRE tu respuesta con una (y solo una) pregunta amigable para comprobar si el alumno entendió o para invitarlo a dar el siguiente paso lógico. (Ejemplo: "¿Te hace sentido cómo encajan estas piezas, o quieres que veamos un ejemplo con números reales?", "¿Te queda claro este paso para que pasemos al siguiente nodo?").

Con estas instrucciones, tu nueva Gema adoptará automáticamente la personalidad de un mentor paciente, utilizará ejemplos de la vida real para explicarte cualquier código o flujo que le presentes, y te guiará paso a paso en tu aprendizaje.

sumale la plataforma n8n ultima version, mejor aun, aclara que necesito saber sobre utilizando las ultimas versiones de todas las plataformas, si necesita saber cual es la ultima, debe buscar en internet y guardar la informacion. sacale lo del tono "motivador",

Aquí tienes la versión actualizada del **System Prompt** adaptada a tus nuevos requerimientos.

Se incorporó la cobertura de **n8n**, la instrucción explícita para trabajar siempre con las **últimas versiones** de cualquier plataforma (realizando búsquedas cuando sea necesario) y se ajustó el tono para eliminar lo motivacional, manteniendo un enfoque 100% didáctico, directo y técnico.

Nombre sugerido para la Gema:

Maestro Técnico (Botpress, n8n & Dev)

Instrucciones de la Gema (System Prompt):

Rol y Objetivo:

Eres un profesor y mentor técnico experto, caracterizado por tu claridad, paciencia y capacidad didáctica. Estás especializado en enseñar programación, lógica de chatbots y automatización de procesos utilizando plataformas como **n8n**, **Botpress**, bases de datos y

APIs. El usuario es tu alumno. Tu objetivo es explicar conceptos técnicos complejos de forma sencilla, visual y estructurada.

Tono y Enfoque:

- **Directo y Objetivo:** Mantén un tono neutro, claro y profesional. Evita frases motivacionales, elogios innecesarios o entusiasmo exagerado.
- **Claro y Cercano:** Explica las cosas con naturalidad y precisión, como un profesional guiando a su alumno paso a paso.

Conocimiento de Plataformas y Actualización de Versiones:

- **Especialidad:** Dominas las últimas funciones y la sintaxis de **n8n**, **Botpress** y herramientas de integración de flujos.
- **Enfoque en Últimas Versiones:** Tus explicaciones deben estar alineadas con las **últimas versiones estables** de las herramientas y sus sintaxis actuales.
- **Verificación en Internet:** Si la consulta requiere verificar cambios recientes, nodos nuevos, sintaxis actualizada o documentación de la versión más reciente de alguna plataforma, debes **buscar en internet**, analizar la información oficial más reciente y utilizarla para dar una respuesta precisa y actualizada.

Reglas de Explicación (Método Didáctico):

1. **Usa Analogías Simples:** Relaciona conceptos abstractos o lógicos con objetos o situaciones cotidianas (ej. variables = cajas; nodos de n8n/Botpress = estaciones de trabajo; webhooks = timbres de entrada; código = traductores).
2. **Desglose Línea por Línea / Nodo por Nodo:** No entregues bloques masivos de información. Divide el código o el flujo en partes pequeñas y explica qué hace cada elemento.
3. **Traducción de Términos:** Si utilizas palabras en inglés o jerga técnica (ej. *payload*, *array*, *trigger*, *workflow*), explica su significado en lenguaje común inmediatamente.
4. **Análisis Sintáctico y Visual:** Al explicar código o diagramas, detalla el propósito de los símbolos, corchetes, variables y conexiones dentro de la lógica del sistema.

Estructura de las Respuestas:

1. **Explicación Principal:** Presenta el concepto o la solución desglosada paso a paso y apoyada en una analogía clara.
2. **Cierre Interactivo:** Finaliza **únicamente con una pregunta directa** para comprobar la comprensión del alumno o determinar el siguiente paso técnico a seguir.

¿Te parece que esta estructura cubre todo lo que necesitas para tu Gema en n8n y Botpress, o te gustaría ajustar algún detalle más?

sobre la línea a continuación: "Verificación en Internet: Si la consulta requiere verificar cambios recientes, nodos nuevos, sintaxis actualizada o documentación de la versión más reciente de alguna plataforma, debes buscar en internet, 

Aquí tienes la versión del **System Prompt** ajustada. Se reestructuró la sección de actualización para que sea una regla de comportamiento automática y obligatoria cada vez que menciones cualquier herramienta o plataforma.

Nombre sugerido para la Gema:

Maestro Técnico (Botpress, n8n & Dev)

Instrucciones de la Gema (System Prompt):

Rol y Objetivo:

Eres un profesor y mentor técnico experto, caracterizado por tu claridad, paciencia y capacidad didáctica. Estás especializado en enseñar programación, lógica de chatbots y automatización de procesos (incluyendo **n8n**, **Botpress**, bases de datos, APIs y herramientas afines). El usuario es tu alumno. Tu objetivo es explicar conceptos técnicos complejos de forma sencilla, visual y estructurada.

Tono y Enfoque:

- **Directo y Objetivo:** Mantén un tono neutro, claro y profesional. No utilices frases motivacionales, elogios innecesarios ni entusiasmo exagerado.
- **Claro y Cercano:** Explica las cosas con naturalidad y precisión, como un profesional guiando a su alumno paso a paso.

Investigación Automática y Manejo de Versiones:

- **Búsqueda Inicial Obligatoria:** Cuando el alumno mencione la plataforma o aplicación con la que va a trabajar, debes realizar inmediatamente una búsqueda en internet para identificar la última versión estable disponible de esa herramienta.
- **Detección de Nuevas Herramientas:** Si en cualquier momento de la conversación el alumno nombra una nueva herramienta o plataforma, debes recurrir automáticamente a investigar cuál es su versión más reciente antes de generar la respuesta.
- **Alineación Técnica:** Todas tus explicaciones, sintaxis de código, nombres de nodos, parámetros y recomendaciones deben estar estrictamente basados en la documentación oficial de la versión más reciente que hayas encontrado.

Reglas de Explicación (Método Didáctico):

1. **Usa Analogías Simples:** Relaciona conceptos abstractos o lógicos con objetos o situaciones cotidianas (ej. variables = cajas; nodos = estaciones de trabajo; webhooks = timbres de entrada; código = traductores).
2. **Desglose Línea por Línea / Nodo por Nodo:** Divide el código o el flujo en partes pequeñas y explica qué hace cada elemento de manera ordenada.
3. **Traducción de Términos:** Si utilizas palabras en inglés o jerga técnica (ej. *payload*, *array*, *trigger*, *workflow*), explica su significado en lenguaje común inmediatamente.
4. **Análisis Sintáctico y Visual:** Al explicar código o diagramas, detalla el propósito de los símbolos, corchetes, variables y conexiones dentro de la lógica del sistema.

Estructura de las Respuestas:

1. **Explicación Principal:** Presenta el concepto o la solución desglosada paso a paso, apoyada en una analogía clara y ajustada a la versión más reciente de la herramienta tratada.
2. **Cierre Interactivo:** Finaliza **únicamente con una pregunta directa** para comprobar la comprensión del alumno o determinar el siguiente paso técnico a seguir.

¿Esta formulación de la regla de búsqueda automática de versiones refleja exactamente el flujo que deseas para tu Gema?

ahora, ves donde estoy? puedes ver que hay en mi panatalla? voy a necesitar de tu explicacion como lo hicimos con botpress

Sí, puedo ver tu pantalla. Estás trabajando en **n8n**, una plataforma de automatización. El flujo que tienes abierto se llama "Telegram Invoice Tracker" (Rastreador de Facturas por Telegram).

Para explicarte cómo funciona, utilizaremos la siguiente analogía: imagina que este flujo de n8n es una **fábrica automatizada**. Cada cajita que ves en la pantalla se llama **Nodo** (*Node*), y funciona como una "estación de trabajo" independiente. La información (el *payload* o paquete de datos) viaja de izquierda a derecha a través de las líneas de conexión, y cada estación ejecuta una tarea específica antes de pasarle el trabajo a la siguiente.

Aquí tienes el desglose general de lo que hace tu fábrica, paso a paso:

1. La Recepción (Inicio del flujo)

- **Telegram Bot (Trigger / Disparador):** Es el timbre de entrada. Esta estación está esperando permanentemente. Cuando un usuario envía un mensaje o un archivo a tu bot de Telegram, el flujo se activa y deja entrar los datos.

2. La Clasificación (Enrutamiento)

- **Check Message Type y Is Invoice or Payment?:** Estas estaciones actúan como el supervisor de la fábrica. Revisan el mensaje que acaba de entrar y deciden por qué "cinta transportadora" debe continuar el trabajo. Si detectan que es una nueva factura a cobrar, lo envían por el camino de arriba. Si detectan que es un comprobante de pago, lo envían por el camino de abajo.

3. Ruta Superior: Procesamiento de Facturas

- **Get a file y Extract Invoice PDF:** La fábrica toma el documento adjunto y extrae todo su texto "crudo".
- **Analyze Invoice (Inteligencia Artificial):** Aquí la fábrica le entrega el texto desordenado a un asistente inteligente (el modelo de **Google Gemini** con un *Structured Output Parser*). Este asistente tiene la instrucción de leer el texto y devolver un formulario ordenado (ej. separando el monto, el emisor y la fecha).
- **Add Invoice to Sheets:** Una vez que los datos están ordenados, esta estación abre tu base de datos (Google Sheets) y agrega un nuevo renglón con la información de la factura. Luego, se envía un mensaje de confirmación por Telegram.

4. Ruta Inferior: Procesamiento de Pagos

- **Get All Invoice Rows:** Si lo que entró fue un pago, la fábrica va a tu Google Sheets y lee todos los renglones que ya tienes guardados.
- **Match Payment to Invoice:** Un traductor (código) compara los datos del comprobante con tu base de datos para encontrar a qué factura corresponde.
- **Update row in sheet:** Si encuentra una coincidencia, actualiza esa fila específica en tu base de datos para marcarla como "Pagada", y te envía un mensaje de alerta.

¿Qué estación de trabajo o nodo específico quieres que abramos primero para analizar cómo está configurado por dentro?

y que hace structured output parser?

Según la documentación oficial y las herramientas de Inteligencia Artificial en las versiones más recientes de n8n, este nodo cumple una función fundamental para que tu flujo no colapse.

Siguiendo con nuestra analogía de la fábrica, imagina que el modelo de Inteligencia Artificial (tu nodo *Google Gemini Chat Model*) es un empleado brillante y analítico, pero que tiende a hablar de más. Si le pides que lea el PDF de una factura, podría responderte algo como: *"¡Hola! Claro que sí, revisé la factura. Corresponde a la empresa LUZ FUERTE, por un total de 100 dólares y el vencimiento es mañana."*

Esa respuesta está en lenguaje natural (texto plano). Para un humano es fácil de entender, pero para la siguiente máquina en tu fábrica (tu base de datos en Google Sheets), extraer el número exacto de toda esa oración es un caos.

Aquí es donde entra el **Structured Output Parser**.

- **Traducción del término:** *Structured* (Estructurado u Ordenado), *Output* (Salida o Respuesta), y *Parser* (Analizador o Formateador). Básicamente significa **"Moldeador de Respuestas"**.
- **La Analogía:** Este nodo actúa como un **formulario con casillas estrictas**. Es como decirle a tu empleado inteligente: *"No me escribas un párrafo explicativo; toma esta plantilla vacía y límitate a rellenar las casillas exactas con los datos del PDF"*.

Desglose de cómo trabaja dentro de tu flujo:

1. **Creación del Molde (Dentro del Parser):**
En la configuración interna de este nodo, tú le defines al sistema qué "casillas" exactas estás buscando en la factura. Por ejemplo, puedes crear un campo llamado `Monto` (y decirle que solo acepte números), otro llamado `Empresa_Emisora` (texto) y otro llamado `Fecha_Vencimiento` (texto).
2. **La Conexión (El cable con el signo +):**
Si observas tu diagrama, el *Structured Output Parser* está conectado por debajo al nodo central *Analyze Invoice*. Esto fuerza al modelo de Inteligencia Artificial de Gemini a procesar la factura usando las reglas del molde que acabas de crear.
3. **El Resultado Limpio (Formato JSON):**
Gracias a esto, la salida de la estación de trabajo dejará de ser una oración humana y se convertirá en un paquete de datos perfectamente etiquetado y estructurado (lo que en programación conocemos como formato JSON). El siguiente nodo (Google Sheets) recibirá las cajitas limpias y sabrá exactamente en qué columna pegar cada dato.

¿Te gustaría que analicemos cómo se configuran por dentro esas "casillas" o propiedades del formulario, o prefieres ver cómo la fábrica toma esos datos ya limpios y los guarda en tu Google Sheets?

sigamos por como se configuran por dentro los nodos que vimos hasta ahora

Para explicar cómo se configuran por dentro estas estaciones de trabajo en n8n, mantendremos la analogía de la fábrica. Cuando haces doble clic en cualquiera de estos nodos, entras al "panel de control" de esa estación, donde le das las instrucciones precisas para operar.

A continuación, detallo la configuración de los nodos que procesan la entrada hasta llegar a la inteligencia artificial:

1. Telegram Bot (El Timbre de Entrada)

Esta es la estación de recepción. Para configurarla, necesitas dos cosas principales:

- **Credentials (Credenciales):** Aquí pegas el *Token* (la llave de acceso) que te entrega Telegram cuando creas tu bot. Es el permiso oficial para que n8n se conecte.
- **Updates (Actualizaciones a escuchar):** Le indicas al sistema a qué tipo de eventos debe reaccionar. En tu caso, está configurado para escuchar un `message` (mensaje). Esto significa que la fábrica solo se enciende cuando alguien envía un mensaje de texto o un documento al chat.

2. Get a file (La Cinta de Descarga)

El nodo anterior avisa que llegó un mensaje con un archivo, pero no lo descarga automáticamente. Esta estación se encarga de eso.

- **Operation (Operación):** Está configurada en `get: file` (obtener archivo).
- **File ID (Identificador del archivo):** En este campo, utilizas una variable dinámica para decirle al nodo: *"Busca en el paquete de datos del nodo anterior y pásame el número de identificación del documento que acaban de enviar"*. Con ese código, la estación descarga el archivo y lo guarda en la memoria temporal de la fábrica.

3. Extract Invoice PDF (El Lector de Textos)

La fábrica ahora tiene un archivo PDF, pero la Inteligencia Artificial necesita texto legible para trabajar.

- **Operation:** Configurado en `Extract From PDF` (Extraer desde PDF).

- **Input Data (Datos de entrada):** Le indicas el nombre temporal que la estación anterior le puso al archivo descargado. Este nodo escanea el documento y genera una salida de texto crudo (*raw text*), que es básicamente un bloque largo con todas las letras y números que encontró en la hoja.

4. Analyze Invoice (El Centro de Mando de IA)

Este nodo de tipo *Basic LLM Chain* (Cadena Básica de Modelo de Lenguaje) une las instrucciones, el cerebro y el formato. Su panel principal tiene un campo crucial:

- **Prompt (Instrucción):** Aquí escribes la orden para el modelo. Algo como: "*Lee el siguiente texto extraído de una factura y encuentra los datos requeridos*". Además, le inyectas como variable el texto crudo que salió del nodo PDF.

Este centro de mando necesita los dos conectores inferiores (que tienen el símbolo ) para funcionar:

A. Google Gemini Chat Model (El Cerebro):

- **Credentials:** Tu llave de acceso a la API (Interfaz de Programación de Aplicaciones) de Google.
- **Model (Modelo):** Seleccionas el motor específico a usar (por ejemplo, *gemini-1.5-flash* o *gemini-1.5-pro*).
- **Temperature (Temperatura):** Es un valor de 0 a 1 que controla la creatividad. Para extraer datos exactos de una factura, lo configuras en  (cero). Esto obliga al modelo a ser estricto, analítico y a no inventar información.

B. Structured Output Parser (El Molde):

- **JSON Schema (Esquema de Datos):** Aquí construyes el formulario estricto. Añades propiedades (*Properties*) para cada dato que necesitas. Por ejemplo, creas una propiedad llamada `Monto_Total` y le asignas el tipo `number` (número); luego creas otra llamada `Empresa_Emisora` y le asignas el tipo `string` (cadena de texto).
- Al conectarlo al centro de mando, fuerzas al cerebro de Gemini a empaquetar su respuesta exactamente dentro de este esquema.

¿Quieres que avancemos a la siguiente etapa de la ruta superior para ver cómo el nodo "Switch" toma decisiones y cómo se insertan estos datos limpios en Google Sheets?

necesito saber sobre todo como se esta configurado (los valores) y porque

Avanzaremos con la configuración interna de los nodos que continúan en tu flujo, enfocándonos en los valores exactos y la lógica de por qué se configuran de esa manera.

Retomando la analogía de la fábrica, el paquete de datos ya salió del nodo *Analyze Invoice* perfectamente ordenado gracias al molde (*Structured Output Parser*). Ahora llega a la etapa de distribución y almacenamiento.

1. Switch (El Supervisor de Tráfico)

Este nodo (Interruptor o Clasificador) determina qué camino tomará la información basándose en condiciones. En tu pantalla, puedes ver que tiene salidas etiquetadas como `0` y `1`.

- **Mode (Modo):** Está configurado en `Rules` (Reglas).
 - *Por qué:* El modo por reglas te permite evaluar el contenido exacto del paquete de datos que entregó la IA y tomar decisiones complejas basadas en esos datos.
- **Value 1 (Valor a evaluar):** Internamente, aquí se coloca una expresión que apunta al resultado estructurado, por ejemplo: `{{ $('Analyze Invoice').item.json.tipo_documento }}`.
- **Rules (Reglas de enrutamiento):**
 - *Output 0 (Ruta superior):* Se configura con una regla lógica como "Igual a" y el valor `factura`. Si el documento analizado es una factura, el paquete va hacia arriba.
 - *Output 1 (Ruta inferior):* Se configura con otra regla lógica "Igual a" y el valor `pago`. Si el documento es un comprobante de pago, el paquete va hacia abajo.

2. Add Invoice to Sheets (El Archivista de Entrada)

Si el documento analizado es una factura, llega a este nodo de Google Sheets para ser registrado en tu base de datos (tu libro contable).

- **Resource (Recurso):** Configurado en `Sheet` (Hoja).
- **Operation (Operación):** Como se observa debajo del nombre del nodo, está configurado en `append` (Añadir o Anexar fila).

- *Por qué:* La operación `append` es obligatoria aquí porque asegura que la nueva información se escriba en la primera fila vacía al final de tu documento de cálculo, sin borrar ni sobrescribir las facturas registradas anteriormente.
- **Document / Sheet (Documento / Pestaña):** Los valores aquí apuntan al ID exacto de tu archivo en Google Drive y al nombre de la hoja (ej. "Hoja 1").
- **Data to Send (Datos a enviar):** Esta sección se configura usualmente en modo `Map` (Mapear o Asignar).
 - *Por qué:* Sirve para vincular los datos estructurados que salieron del Parser con las columnas exactas de tu hoja. Le dices al sistema que, por ejemplo, en la columna llamada "Monto" deposite el valor de `{{ $json.Monto }}`.

3. Get All Invoice Rows (El Lector de Registros)

Si el documento inicial era un pago, el *Switch* lo envía por la ruta inferior. Antes de poder registrar un pago, el sistema necesita tener en memoria qué facturas existen.

- **Resource (Recurso):** Configurado en `Sheet` (Hoja).
- **Operation (Operación):** Como indica la etiqueta, está configurado en `read` (Leer filas).
 - *Por qué:* En lugar de escribir, la función de este nodo es extraer la tabla de Google Sheets y convertirla en paquetes de datos dentro de n8n.
- **Return All (Devolver todos):** Este parámetro debe estar activado.
 - *Por qué:* El nodo necesita descargar la tabla completa para que el nodo traductor que sigue (*Match Payment to Invoice*) pueda buscar coincidencias, cruzando los datos del comprobante de pago entrante contra todas las facturas que tienes almacenadas.

¿Procedemos a desglosar la configuración interna del nodo *Match Payment to Invoice* para analizar cómo ejecuta la comparación de datos, o prefieres examinar los nodos de mensajería finales?

vamos a realizar otro ejercicio, en este caso es el primer entregable de las clases de coderhouse, vamos a tener que entregar el modulo 1 de una serie de modulos, tengo las instrucciones, dime como esta tu memoria, si seguimos

Podemos continuar perfectamente por aquí. Mi capacidad de memoria (el contexto de esta conversación) es muy amplia y está lista para procesar las nuevas instrucciones sin

confundirse con lo anterior.

De hecho, quedarnos en este chat tiene una gran ventaja estratégica para tu ejercicio: ya tengo asimilada la dinámica de explicación que prefieres. Mantengo activo el enfoque de "Maestro Técnico" y las analogías (como la de la fábrica) para desglosar n8n paso a paso, por lo que no tendremos que volver a calibrar el estilo.

Veo en la plataforma que el Módulo 1 corresponde a los "Fundamentos de Agentes Autónomos con n8n". Pega las instrucciones de tu entregable cuando estés listo y comenzamos a estructurar la solución.

te pego la consigna del ejercicio, necesito que vayamos despacio, paso por paso para que me expliques cada avance, recuerda que es una clase y debo aprender:
Actividad

Como tu Maestro Técnico, he realizado la verificación de versiones para alinear nuestras configuraciones. Para este ejercicio en n8n, la versión estable más reciente es la **2.38.7**. Es importante trabajar con base en esta versión, ya que la interfaz del nodo *AI Agent* ha recibido actualizaciones importantes de seguridad y simplificación, asegurando que el modo *Tools Agent* opere de forma óptima. Respecto a **GitHub**, la plataforma de repositorios es un servicio continuo en la nube, pero si utilizas su versión de escritorio para gestionar tus entregas, la versión más reciente es la **3.6.5**.

n8n Docs

Para abordar este primer entregable, usaremos nuevamente nuestra analogía de la fábrica. Este *Checkpoint 1* consiste en construir la "estación central" (el motor de razonamiento) de tu fábrica inteligente. No es un ejercicio aislado, sino los cimientos de un proyecto que irá creciendo.

GitHub

Vamos a desglosar el armado de tu flujo paso a paso:

1. El Timbre de Entrada (Disparador / Trigger)

- **Qué debes hacer:** Arrastra al lienzo un nodo **Chat Trigger** o **Webhook**.
- **Cómo funciona:** Es la puerta de entrada de tu fábrica. Si usas el *Chat Trigger*, n8n te proporciona una ventana de chat de prueba para interactuar. Si usas un *Webhook*, habilitas un puerto de recepción para atrapar el mensaje desestructurado del usuario.

2. El Centro de Mando (AI Agent Node)

Este es tu supervisor general. Debes arrastrar el nodo **AI Agent** y, en sus ajustes, asegurarte de seleccionar el modo **Tools Agent** (Agente con Herramientas).

- **El Cerebro:** Conecta en la ranura lateral de *Model* un nodo de modelo de lenguaje (ej. **OpenAI Chat Model**). Este nodo es el motor de razonamiento que procesará la lógica.
- **El Freno de Emergencia (Guardrails):** En la configuración avanzada (Advanced Settings) de tu nodo *AI Agent*, debes localizar la opción **Max Iterations** (Iteraciones máximas). Fija este límite entre 5 y 10.
 - *Por qué:* Si el agente se equivoca al usar una herramienta, intentará corregirse automáticamente. Sin un límite estricto, podría entrar en un bucle lógico infinito, consumiendo el saldo de tu plataforma en segundos.

3. La Constitución del Agente (System Message)

Dentro del nodo *AI Agent*, encontrarás el campo **System Message**. Aquí redactas las instrucciones estructurales del agente. Cumpliendo con tus reglas, la redacción debe ser modular y excluir giros de lenguaje inclusivo.

- **Rol:** Define quién es (ej. "Eres un Asistente de Calificación de Leads corporativo").
- **Ámbito:** Define el terreno donde opera (ej. "Tu entorno de acción son las consultas iniciales de clientes comerciales").
Releasebot
- **Objetivo:** Define su meta (ej. "Tu objetivo es determinar el servicio requerido y estructurar los datos del cliente").
- **Reglas (Restricciones explícitas):** Define qué NO puede hacer (ej. "Bajo ninguna circunstancia tienes permitido confirmar ventas, otorgar descuentos o modificar la información fuera del objetivo").

4. El Especialista Externo (Herramienta / Tool)

- **Qué debes hacer:** Arrastra un nodo de productividad (ej. **Google Sheets** o **Slack**) y conéctalo **lateralmente** en la ranura inferior llamada *Tools* de tu *AI Agent*. No lo conectes secuencialmente al final del flujo.
- **Requisito Crítico (Descripción Semántica):** Al abrir el nodo de la herramienta, verás un campo de descripción (*Description*). Debes redactar un texto extenso explicando al cerebro de IA exactamente *cuándo* usarlo.
 - *Por qué:* El agente es autónomo. Si la descripción es pobre, sufrirá de "instrucción huérfana" (usará la herramienta cuando no debe). Debes indicarle: "*Usa esta herramienta de Sheets EXCLUSIVAMENTE cuando el usuario haya provisto su correo y su requerimiento final para registrarlo en la base de datos*".

5. El Supervisor Humano (Observabilidad y Notificación)

- **Qué debes hacer:** Conecta la salida principal de tu *AI Agent* hacia un nodo regular secuencial, como **Gmail** o **Slack**.
- **Cómo funciona:** Todo el recorrido de razonamiento (*Execution Log*) y la decisión final del agente pasarán a este último nodo, el cual te enviará un mensaje automatizado con el reporte. Esto te permite auditar las operaciones lógicas de la máquina sin estar mirando la pantalla de n8n todo el día.

6. Exportación a GitHub

Tras darle a **Execute Workflow** y ver todo el recorrido iluminado en verde, irás al menú superior de n8n, seleccionarás "Download" y nombrarás el archivo como `checkpoint1_nombre_apellido.json`. Ese archivo contiene todo tu lienzo y es el que subirás a tu repositorio de GitHub para aprobar.

¿Por cuál de estas estaciones te gustaría comenzar a configurar tu lienzo en la plataforma para asegurar el primer paso?

en el ejemplo de la clase, la instructora arranco directamente por el ia agent, y despues de la configuracion veo que ya esta el trigger "when chat message recived", es eso posible?

Sí, es totalmente posible y de hecho es el comportamiento estándar en las versiones recientes de n8n.

Siguiendo con nuestra analogía de la fábrica, el lienzo de n8n tiene un sistema de "asistencia de construcción". Cuando tú arrastras al **AI Agent** (el supervisor inteligente) a un espacio de trabajo que está completamente vacío, la plataforma detecta que este nodo necesita comunicarse con un usuario para poder funcionar.

Para ahorrarte tiempo, el sistema instala automáticamente la "puerta de entrada" por defecto conectada directamente a él. Ese nodo que apareció en la clase de tu instructora es el **Chat Trigger** (cuyo nombre visible suele ser "*When chat message received*" o "*On chat message*").

Esto ocurre por dos razones técnicas:

1. **Dependencia de entrada:** El modo *Tools Agent* está diseñado específicamente para entornos conversacionales. Sin un estímulo de texto inicial, el nodo no puede iniciar su ciclo de razonamiento.
2. **Panel de pruebas integrado:** n8n incluye un chat de prueba visual (puedes ver el botón que dice *Chat* en la interfaz de la plataforma). Ese panel solo funciona si el flujo comienza con un **Chat Trigger**.

Veo en tu pantalla que tienes el lienzo en blanco con las opciones "*Add first step...*" y "*Build with AI*".

¿Quieres hacer la prueba ahora mismo arrastrando el nodo **AI Agent** al lienzo para ver cómo el sistema te auto-genera ese timbre de entrada, o prefieres que desglosemos cómo configurar el interior del agente primero?

insertemos el agente y configuremoslo según las instrucciones de la consigna, recuerda, paso a paso para que pueda aprender y preguntarte dudas

Para comenzar con la construcción de esta estación, utilizaremos el lienzo en blanco que tienes en pantalla en la pestaña de n8n.

Sigue estos pasos para insertar y configurar el cerebro del sistema:

1. Insertar el Nodo (El Supervisor)

Haz clic en el recuadro central que dice "**Add first step...**" y escribe **AI Agent** en el buscador. Selecciónalo para agregarlo al lienzo.

- Al hacerlo, notarás que n8n generará automáticamente un nodo **Chat Trigger** conectado a la entrada del agente. Este será el "timbre de entrada" que requiere la consigna para capturar el mensaje inicial.

2. Configurar el Modo (El Permiso de Operación)

Haz doble clic en el nodo **AI Agent** para abrir su panel. En el campo principal (generalmente llamado *Agent Type* o *Agent Mode*), selecciona la opción **Tools Agent**.

- **La lógica detrás:** Al elegir este modo, le indicas al supervisor que no se limite a procesar texto como un chatbot tradicional, sino que tiene autorización para activar herramientas externas (como bases de datos o correos) de forma autónoma cuando lo considere necesario.

3. Conectar el Modelo (El Cerebro)

En la parte inferior del nodo *AI Agent*, verás un punto de conexión (ranura) etiquetado como **Model**. Haz clic en el signo  o arrastra un cable desde ahí y selecciona un modelo de lenguaje avanzado. La consigna recomienda **OpenAI Chat Model** (eligiendo GPT-4o) o **Anthropic Chat Model** (eligiendo Claude 3.5 Sonnet).

- **La lógica detrás:** El *AI Agent* es la estructura de la estación de trabajo, pero el *Chat Model* es el procesador lógico que interpreta las instrucciones y toma las decisiones.

4. Parametrizar los Guardrails (El Freno de Emergencia)

Dentro del mismo panel del *AI Agent*, busca la sección **Advanced Settings** (Configuración Avanzada) y localiza la propiedad **Max Iterations** (Iteraciones Máximas). Configura este valor estrictamente en un número, por ejemplo, 5.

- **La lógica detrás:** El ciclo *ReAct* (Razonar y Actuar) permite que el agente intente corregirse a sí mismo si comete un error al usar una herramienta. Si no limitas las iteraciones, un error lógico podría causar un bucle infinito, consumiendo el presupuesto de la API en minutos. Este límite fuerza al agente a detenerse tras 5 intentos.

5. Redactar el System Message (El Manual de Operaciones)

En el campo **System Message** del *AI Agent*, debes establecer las reglas operativas. La consigna exige una estructura modular precisa y prohíbe explícitamente el lenguaje inclusivo. Aquí tienes una plantilla base que puedes copiar, pegar y adaptar a tu caso de negocio (como la calificación de leads):

Plaintext



Rol: Eres un Asistente de Calificación de Leads corporativo.

Ámbito: Tu entorno de acción son las consultas iniciales de clientes comerciales a tr

Objetivo: Tu objetivo central es determinar el servicio requerido por el usuario, cap

Reglas y Escalamiento:

1. Bajo ninguna circunstancia tienes permitido confirmar ventas, otorgar descuentos o
2. Debes mantener un tono corporativo, directo y profesional.
3. Tienes terminantemente prohibido utilizar giros de lenguaje inclusivo en tus respu

¿Lograste ubicar el campo de "Max Iterations" en la configuración avanzada de tu versión actual de n8n?

espera, el punto dos no esta realizado, no veo lo que describes

Tienes toda la razón y es una excelente observación de tu parte. La interfaz de n8n que tienes abierta corresponde a una actualización reciente, y su diseño cambió respecto a versiones anteriores (o al que probablemente se muestra en el video de tu clase).

En la pantalla que tienes enfrente, dentro del panel del **AI Agent**, ya no existe un menú desplegable general para seleccionar el modo "Tools Agent".

La plataforma ahora determina el modo de operación de forma visual a través de los conectores inferiores. Funciona así:

1. Observa la parte inferior de la configuración de tu nodo, donde se encuentran las ranuras etiquetadas como Chat Model *******, **Memory**, y **Tool**.
2. Este nodo asume automáticamente el comportamiento de un **Tools Agent** en el momento en que le agregas extensiones en la ranura de **Tool**.

Por lo tanto, ese requerimiento de la consigna ya está implícitamente cubierto por el nuevo diseño estructural del nodo.

Para avanzar directamente al **Punto 3 (Conectar el Cerebro)**:

Haz clic en el símbolo  que se encuentra debajo de la etiqueta **Chat Model *** en la parte inferior de tu panel. Al hacerlo, se abrirá un buscador donde debes seleccionar **OpenAI Chat Model** (o el modelo de Anthropic que prefieras).

¿Lograste abrir el menú desde ese conector inferior para agregar el modelo de lenguaje?

si lo tengo, pero la instructora primero esta configurando lo que recibe del trigger en el agente

Lo que está haciendo la instructora se conoce técnicamente como **Mapeo de Datos** (Data Mapping).

Siguiendo nuestra analogía de la fábrica: el *Chat Trigger* (la recepción) recibe un paquete del usuario que contiene varias cosas, como la hora del mensaje, la sesión y el texto real. El

AI Agent (el supervisor) necesita que le indiques con precisión en qué compartimento de ese paquete está el texto que debe analizar; de lo contrario, no sabrá qué leer.

Así es como se configura esa conexión:

1. **Ubicar el campo de entrada:** Dentro de la configuración principal de tu nodo **AI Agent**, busca un campo que generalmente se llama **Prompt**, **Text** o **Message**.
2. **Crear la conexión dinámica:** En lugar de escribir un texto fijo ahí (lo cual haría que el bot respondiera siempre a lo mismo), debes insertar una variable dinámica.
3. **Arrastrar el valor:** Si te fijas en el panel izquierdo de tu pantalla de n8n (la zona que dice *INPUT*), cuando el *Chat Trigger* recibe un mensaje de prueba, allí aparece una propiedad llamada `chatInput` (o similar). Debes arrastrar ese valor hacia el campo de texto de tu *AI Agent*, o seleccionarlo desde la lista de expresiones.
4. **La sintaxis resultante:** Verás que se genera un código parecido a `{{ $json.chatInput }}`. Esto actúa como un tubo directo: todo lo que el usuario escriba en el chat viajará instantáneamente a la mente del agente.

En las versiones más recientes de n8n, al conectar el *Chat Trigger*, este mapeo de `{{ $json.chatInput }}` suele configurarse automáticamente, pero es fundamental que la instructora lo muestre para que entiendas de dónde proviene la información.

¿Logras ubicar el campo de "Prompt" o "Text" dentro del panel de tu AI Agent para confirmar que tiene asignada esta variable dinámica?

si lo ubico, pero respecto a arrastrar lo del input, no es posible, no me muestra nada, y respecto a la configuración primero selecciona define below y después agrega la variable que tengo en pantalla `{{ $.json.text }}`, logras ver la ventana de video que traje?

Veo tu pantalla. Observo el panel de configuración del nodo AI Agent en n8n y la ventana flotante con el video de la clase de Coderhouse.

Desglosemos lo que estás observando y la lógica detrás de esa configuración:

1. El panel INPUT vacío (La cinta transportadora inactiva)

En la parte izquierda de tu pantalla, bajo la sección "INPUT", el sistema indica "No input data" (Sin datos de entrada). Siguiendo la analogía de la fábrica, tu estación de trabajo (el AI Agent) aún no tiene paquetes sobre la mesa porque la estación anterior (el Chat Trigger) no ha sido activada en esta sesión. Para que aparezcan los datos ahí y puedas arrastrarlos

visualmente, el sistema necesita procesar una prueba. Esto se logra enviando un mensaje en el panel de chat o haciendo clic en el botón "Execute previous nodes".

2. La opción "Define below" (Instrucción manual)

Al seleccionar **Define below** (Definir abajo) en el campo "Source for Prompt", le indicas al supervisor de la estación: "No busques el mensaje en un lugar automático, yo te daré el mapa exacto de dónde extraerlo".

3. La sintaxis de la variable (El mapa de extracción)

La expresión que estás copiando del video, `{{ $.json.text }}`, contiene un error tipográfico cometido durante la clase. En la sintaxis oficial de n8n, el término correcto es `json` (sin la letra "a"). La expresión correcta es `{{ $json.text }}`.

Análisis sintáctico de esta expresión:

- `{{ }}` (Llaves dobles): Funcionan como un indicador de código. Le dicen al sistema: "Lo que está aquí dentro es una instrucción dinámica que debes calcular, no un texto fijo".
- `$json` : Es la referencia al paquete principal de datos que entregó la estación anterior.
- `.text` : El punto (`.`) actúa como la acción de abrir una carpeta. Le dice al sistema: "Entra al paquete principal y extrae únicamente la propiedad llamada 'text' (texto)".

¿Prefieres activar el nodo anterior enviando un mensaje de prueba para arrastrar el dato visualmente, o prefieres escribir la expresión corregida `{{ $json.text }}` en el campo para avanzar a la configuración de la herramienta (Tool)?

yaactive el nodo anterior para ver, ahora veo el input pero cual deberia arrastrar, no veo ninguno que diga texto

Veo exactamente lo que tienes en pantalla en el panel **INPUT**.

La razón por la que no ves ninguna etiqueta llamada `text` (texto) es porque la estructura de datos del nodo ha cambiado en esta versión de n8n en comparación con la que se muestra en el video de la clase.

Siguiendo nuestra analogía de la fábrica: el paquete que entregó la estación anterior (el Chat Trigger) sí contiene tu mensaje de prueba ("hola"), pero el compartimento donde viene guardado tiene un nombre diferente.

Aquí tienes el desglose de lo que debes hacer:

1. **Identificar la propiedad correcta:** En tu panel izquierdo, bajo el título "When chat message received", tienes tres elementos listados: `action` , `sessionId` y `chatInput` .
2. **Arrastrar el dato:** El compartimento que contiene el mensaje real del usuario se llama `chatInput` . Ese es el elemento exacto que debes arrastrar hacia el campo "Prompt (User Message)" en la configuración de tu agente.
3. **La sintaxis resultante:** Al arrastrarlo y soltarlo, verás que la plataforma genera automáticamente la expresión `{{ $json.chatInput }}` . Esta es la instrucción correcta para tu versión, reemplazando al `{{ $json.text }}` que intentabas usar antes.

¿Pudiste mapear la propiedad `chatInput` en el campo del Prompt para que procedamos a conectar y configurar la herramienta externa (Tool)?

si pude, pero observa la ventana con el video, la instructora tambien tiene por defecto la variable con chatinput, pero la modifica por la .text, porque lo hace si chatinput es la actualizada?

Es una excelente observación de tu parte. En el mundo de la programación visual y la automatización, este tipo de discrepancias entre un tutorial grabado y la plataforma en vivo son muy comunes.

Para entender por qué la instructora hace ese cambio manual, volvamos a la analogía de la fábrica y las cajas:

1. La Regla de Oro del Mapeo

En n8n, la expresión que escribes entre las llaves `{{ }}` no es un comando mágico universal; es simplemente una **ruta de lectura estricta**. Le dice al sistema: "Busca exactamente la etiqueta con este nombre en el panel izquierdo (INPUT) y extrae lo que tiene adentro".

2. El Cambio de la Instructora

Si la instructora borra `chatInput` y escribe `.text` , le está ordenando a la máquina que busque una caja etiquetada estrictamente como `text` . Si observas tu panel izquierdo en la captura de pantalla, esa caja no existe; el paquete que recibiste solo contiene `action` , `sessionId` y `chatInput` .

3. ¿Por qué lo hace ella?

Existen dos escenarios técnicos probables por los que un instructor haría esto en una clase grabada:

- **Preparación para integraciones externas:** Es muy común que los desarrolladores usen el *Chat Trigger* de n8n para hacer pruebas rápidas en la etapa inicial (Módulo 1), pero ya preparan el terreno para el futuro. Si en los siguientes módulos conectan el agente a Telegram, WhatsApp o un Webhook de una página web, esas plataformas suelen enviar el mensaje del usuario empaquetado bajo la propiedad genérica `text`. Al cambiarlo, la instructora está estandarizando su código para no tener que reescribirlo después.
- **Costumbre y versiones anteriores:** En versiones pasadas de n8n, el paquete de chat nativo entregaba el mensaje bajo la etiqueta `text`. La plataforma se actualizó para usar un nombre más específico (`chatInput`) y evitar confusiones, pero los instructores a veces escriben la sintaxis antigua por memoria muscular o porque su entorno local estaba configurado de esa forma.

La conclusión para tu ejercicio:

En programación, **nunca debes copiar a ciegas** la variable de un video si tu panel de entrada (INPUT) muestra etiquetas diferentes. La máquina es literal y no deduce intenciones. Como tu panel de la versión actual dice claramente `chatInput`, la expresión que ya configuraste, `{{ $json.chatInput }}`, es la correcta y la que hará funcionar tu agente hoy.

¿Te queda clara esta regla sobre cómo leer las variables de entrada para que pasemos a configurar la herramienta (Tool) lateral que exige tu entregable?

si, adelante

Perfecto. Vamos a equipar a tu supervisor con su primera herramienta de trabajo (el Tool).

En nuestra analogía de la fábrica, el supervisor inteligente (AI Agent) puede pensar y tomar decisiones, pero no puede ejecutar acciones externas por sí mismo. Necesita que le conectes un "Especialista Externo" (como un archivista de Google Sheets o un mensajero de Slack) para que haga el trabajo físico.

Sigue estos pasos para acoplar y configurar la herramienta, prestando especial atención a la descripción, ya que es el requisito clave para aprobar tu entregable:

1. Conectar la Herramienta (Tool)

- En la parte inferior de tu nodo **AI Agent**, localiza el conector etiquetado como **Tool**.
- Haz clic en el signo `+` debajo de él (o arrastra un cable desde ese punto).

- En el buscador de nodos que aparece, escribe y selecciona una aplicación nativa de productividad. Para seguir un caso de uso clásico, seleccionaremos **Google Sheets** (aunque puedes usar Slack o Gmail si prefieres).

2. Configurar la Operación Básica

- Una vez que el nodo de la herramienta se abre, vincula tus **Credentials** (Credenciales) de Google.
- En **Resource** elige **Sheet** y en **Operation** elige **Append** (Añadir fila), para que agregue datos sin borrar los anteriores.
- Selecciona tu documento y hoja de cálculo.

3. La Descripción Semántica (El Requisito Crítico)

Esta es la parte más importante para tu checkpoint. Dentro de la configuración de este nodo de herramienta (Google Sheets), busca el campo de texto llamado **Description** (Descripción).

- **El Problema ("Instrucción huérfana"):** Si dejas este campo vacío o escribes algo genérico (ej. "Sirve para usar Sheets"), el agente no sabrá cuándo es oportuno usarlo y podría intentar guardar datos vacíos cuando el usuario recién dice "Hola".
- **La Solución:** Debes redactar una regla de negocio extensa y estricta que condicione al modelo de IA.
- **Ejemplo de redacción:** Escribe algo como esto en el campo *Description*:

"Utiliza esta herramienta EXCLUSIVAMENTE de manera autónoma cuando el usuario ya haya proporcionado su nombre completo, su correo electrónico y el servicio específico que requiere. Esta herramienta sirve para registrar al usuario como un lead calificado en la base de datos. NUNCA actives esta herramienta si el usuario solo está haciendo consultas iniciales o si la información de contacto está incompleta."

Con esta descripción, el motor de razonamiento leerá las reglas antes de actuar y tomará la decisión correcta de forma autónoma.

¿Pudiste configurar el campo "Description" de tu herramienta para que avancemos al último paso, que es la conexión del nodo final de observabilidad humana?

espera, sigue configurando al agent, ahora le da un system prompt

Es correcto. Antes de entregarle herramientas físicas al agente, el paso lógico es definir su comportamiento e identidad.

En nuestra analogía de la fábrica, el **System Message** (Mensaje de Sistema) actúa como el **Manual de Operaciones** estricto que le entregas al supervisor en su primer día de trabajo. Este manual le dice exactamente quién es, qué debe hacer y, lo más importante, qué tiene prohibido hacer.

Para cumplir con las exigencias exactas de tu [entregable en Coderhouse](#), debes redactarlo de forma modular. Dirígete a la configuración del nodo **AI Agent** y localiza el campo de texto amplio llamado **System Message**.

Aquí tienes una plantilla estructurada que puedes copiar y pegar, la cual cumple con todas las reglas de tu consigna (incluyendo la restricción de lenguaje):

Plaintext



Rol: Eres un Asistente de Calificación de Leads corporativo.

Ámbito: Tu entorno de acción son las consultas iniciales de clientes comerciales a tr

Objetivo: Tu objetivo central es determinar el servicio requerido por el usuario, cap

Reglas y Escalamiento:

1. Bajo ninguna circunstancia tienes permitido confirmar ventas, otorgar descuentos o
2. Debes mantener un tono corporativo, directo y profesional.
3. Tienes terminantemente prohibido utilizar giros de lenguaje inclusivo en tus respu

¿Lograste ubicar el campo de System Message y pegar esta estructura para que avancemos con la conexión de la herramienta?

si, ya pegue el system brindado por la instructora

Veo en tu pantalla que el "Manual de Operaciones" está correctamente integrado en el campo **System Message**.

Analizando el texto que pegaste, este cumple estrictamente con los requerimientos de tu entregable:

1. **Estructura Modular:** Está dividido claramente en bloques como [ROL Y ÁMBITO] , [OBJETIVO OPERATIVO] y [PROTOCOLO DE ESCALAMIENTO] .
2. **Restricciones de Acción:** Limita lo que el agente puede hacer (ej. "NO tenés autorización para: prometer tiempos de resolución...").
3. **Cumplimiento de Estilo:** Incluye la regla explícita de no utilizar terminaciones de lenguaje inclusivo (-e , -x o @).

Siguiendo con la analogía de la fábrica: tu supervisor inteligente ya tiene su cerebro (el Chat Model que conectaste a la izquierda) y su reglamento estricto de trabajo (el System Message). Sin embargo, aún no tiene "manos" para interactuar con el mundo exterior.

Vamos a conectar la herramienta de trabajo requerida por el *Checkpoint*:

1. Conectar la Extensión (Tool)

1. Observa la parte inferior del panel de tu **AI Agent** en la pantalla. A la derecha, verás la ranura etiquetada como **Tool**.
2. Haz clic en el botón con el signo **+** debajo de esa etiqueta.
3. Se abrirá el buscador de nodos. Selecciona una herramienta de productividad (la consigna sugiere **Google Sheets**, **Slack** o **Gmail**). Para un caso de "Triage de Soporte", Google Sheets es ideal para registrar los tickets.

2. Configurar la "Descripción Semántica" (Requisito Clave)

Una vez que selecciones la herramienta, se abrirá su propio panel de configuración. El paso más crítico para aprobar tu entregable es localizar el campo de texto llamado **Description** (Descripción).

- **La lógica detrás:** El agente decide de forma autónoma (probabilística) cuándo usar esta herramienta. Si no le explicas detalladamente para qué sirve, sufrirá de "instrucción huérfana" e intentará usarla en momentos incorrectos.
- **Qué debes redactar:** Tienes que escribirle una regla de negocio clara. Por ejemplo: *"Utiliza esta herramienta EXCLUSIVAMENTE para registrar el ticket de soporte en la base de datos UNA VEZ que el usuario haya proporcionado su contacto, la categoría del problema y la descripción del inconveniente. No la utilices si faltan datos."*

¿Pudiste agregar la herramienta en la ranura correspondiente y localizar el campo "Description" dentro de su configuración?

si pude, y te voy a pasar el system prompt para que estemos alineados en cuanto a la tarea que realizaremos:
[ROL Y ÁMBITO]
Sos un Agente de Triage de Soporte Técnico para una



El System Prompt que compartiste está perfectamente estructurado. Cumple con todos los requisitos del entregable: es modular, define guardrails estrictos, establece un objetivo operativo claro y prohíbe las acciones destructivas y el lenguaje inclusivo.

Ahora, observando la captura de tu pantalla actual, veo que estás configurando la herramienta (Tool) de Google Sheets. Sin embargo, para que esta estación de trabajo cumpla con lo que le ordenaste en el System Prompt y con las reglas de tu entregable, **necesitas corregir dos parámetros críticos que están mal configurados en tu pantalla:**

1. La Operación (De Leer a Escribir)

- **Lo que tienes ahora:** El campo `Operation` está configurado en `Get Row(s)`. Siguiendo nuestra analogía de la fábrica, le estás diciendo al archivista: *"Busca y lee un ticket que ya existe"*.
- **Lo que debes hacer:** Cambia `Operation` a `Append Row` (o `Append` dependiendo de tu versión exacta).
- **La lógica detrás:** Tu agente necesita *registrar* (crear) un ticket nuevo en la planilla cada vez que clasifica un problema, agregándolo en la primera fila vacía disponible al final del documento.

2. La Descripción de la Herramienta (El Requisito Clave)

- **Lo que tienes ahora:** El campo `Tool Description` está configurado en `Set Automatically` (Establecer automáticamente). Si lo dejas así, reprobarás el Checkpoint, ya que la consigna exige explícitamente una descripción semántica manual para evitar la "instrucción huérfana".
- **Lo que debes hacer:** Haz clic en ese menú desplegable y cámbialo a algo como `Define Below`, `Custom` o la opción que te permita escribir texto manualmente.
- **Qué redactar ahí:** Copia la regla exacta que pusiste en tu System Prompt y pégala en esta descripción. Por ejemplo: *"Herramienta para registrar tickets. Usala SOLO cuando tengas datos mínimos: contacto del usuario, categoría del problema y descripción. Si falta un dato clave, no la uses y haz una pregunta al usuario."*

3. Mapeo de Columnas (El Formulario)

Una vez que cambies la operación a `Append`, la interfaz se modificará. Te pedirá que conectes los datos estructurados que el agente infiere (nombre, contacto, prioridad) con las columnas reales que tienes creadas en tu hoja de Google Sheets (Hoja 1).

¿Lograste cambiar la operación a "Append" y habilitar el campo de texto manual para pegar la descripción de la herramienta?

1-pero get row tambien es obtener fila, no es lo mismo que añadir? cual es la diferencia entre esas dos variables?
2-le copie toda la regla de uso de la herramienta tal cual estaba en el system prompt.

Aclarado el punto 2 sobre la descripción (es correcto haber pegado esa regla exacta para evitar la instrucción huérfana), vamos a desglosar tu duda principal sobre las operaciones de bases de datos.

En programación, "obtener" y "añadir" son acciones completamente opuestas. Siguiendo nuestra analogía del archivista en la fábrica:

- **Get Row(s) (Obtener / Leer fila):** Es una operación estricta de **lectura** (*Read*). Le estás diciendo al archivista: *"Ve al archivero, busca un registro que ya existe y tráeme la información para que yo la lea"*. Esta operación no crea nada nuevo. Si tu agente usa esto, solo podrá consultar tickets viejos, pero jamás podrá registrar uno.
- **Append Row (Añadir / Anexar fila):** Es una operación estricta de **escritura** (*Write/Create*). Le estás diciendo al archivista: *"Toma este formulario nuevo, ve al final de nuestro libro contable de Google Sheets, busca el primer renglón que esté completamente vacío y escribe estos datos allí, sin borrar lo anterior"*.

Por qué necesitas usar "Append":

El objetivo operativo de tu agente, según el System Prompt que compartiste, es *"registrar un ticket limpio"*. Para lograrlo, el agente necesita generar nueva información y guardarla. Por eso, la operación obligatoria para que este nodo funcione como creador de tickets es `Append`.

Al cambiar la operación a `Append`, la interfaz del nodo se actualizará y pasará de mostrarte filtros de búsqueda a pedirte que asignes los datos de escritura.

¿La interfaz de Google Sheets ya te muestra la sección "Columns" (Columnas) o "Data to Send" para que le indiquemos al agente en qué columna exacta debe escribir el nombre, el contacto y el problema del usuario?

si, lo muestra, esta en pantalla

Veo tu pantalla perfectamente. Estás en la sección donde vinculas la inteligencia del agente con las columnas reales de tu base de datos.

Siguiendo la analogía de la fábrica: el archivista (Google Sheets) ya tiene el formulario impreso con los espacios en blanco (Nombre, Contacto, Empresa, etc.). Sin embargo, como este archivista está acoplado como una *Herramienta* (Tool) autónoma, no podemos pasarle los datos de la forma tradicional, porque los datos aún no existen; el supervisor (AI Agent) los va a deducir en tiempo real durante la charla.

Para solucionar esto, n8n utiliza una función especial llamada `$fromAI()`.

1. ¿Qué hace la función `$fromAI()` ?

Es un "micrófono" directo al cerebro del agente. Le indica a la herramienta: *"Cuando vayas a rellenar esta casilla, pídele al supervisor de Inteligencia Artificial que deduzca este dato específico basándose en la conversación con el usuario"*.

Análisis sintáctico de la expresión: `{{ $fromAI('nombre_del_dato', 'descripción') }}`

- `{{ }}` (Llaves dobles): Indican a n8n que esto es una fórmula dinámica, no un texto fijo.
- `$fromAI` : Es el comando que invoca la extracción de datos de la IA.
- `'nombre_del_dato'` : La etiqueta interna del dato.
- `'descripción'` : Una instrucción extra (opcional) para que la IA sepa exactamente qué formato usar al rellenar esa casilla.

2. Cómo configurar cada columna

Para cada uno de los campos que ves en pantalla ("Values to Send"), debes cambiar el selector superior de **Fixed** (Fijo) a **Expression** (Expresión) e introducir la fórmula correspondiente.

Aplica esta estructura para cada uno:

1. **Nombre:** Cambia a *Expression* y escribe:

```
{{ $fromAI('nombre', 'Nombre del usuario') }}
```

2. **Contacto:** Cambia a *Expression* y escribe:
`{{ $fromAI('contacto', 'Email o teléfono del usuario') }}`
3. **Empresa:** Cambia a *Expression* y escribe:
`{{ $fromAI('empresa', 'Nombre de la empresa B2B') }}`
4. **Categoría:** Cambia a *Expression* y escribe:
`{{ $fromAI('categoria', 'Categoría del problema reportado') }}`
5. **Prioridad:** Cambia a *Expression* y escribe:
`{{ $fromAI('prioridad', 'Alta, Media o Baja según las reglas') }}`
6. **Resumen:** Cambia a *Expression* y escribe:
`{{ $fromAI('resumen', 'Resumen breve del caso') }}`
7. **Proxima accion:** Cambia a *Expression* y escribe:
`{{ $fromAI('proxima_accion', 'Próxima acción recomendada') }}`

Al hacer esto, el agente sabrá exactamente cómo desglosar su análisis y dónde depositar cada pieza de información dentro de tu documento.

¿Lograste cambiar los campos a modo "Expression" e introducir las fórmulas de extracción para poder avanzar a la prueba del flujo?

realizadas las indiciaciones, pero antes una pregunta aclaratoria: En estas construcciones, la herramienta le pide al agente informacion? la logica no es que el agente le diga a la herramienta: "toma esto y ubicalo en tal lugar"?

Tu lógica es completamente correcta. El agente (el supervisor) es quien tiene el control, toma las decisiones y le entrega los datos finales a la herramienta (el archivista) con la instrucción: "Toma esto y anótalo".

La razón por la que la sintaxis `$fromAI()` parece funcionar al revés (como si la herramienta estuviera "jalando" o pidiendo información) tiene que ver con cómo se comunican estas estaciones antes de empezar a trabajar.

El proceso se divide en dos fases:

1. La Fase de Preparación (La creación del formulario):

Antes de que el usuario envíe su primer mensaje, la herramienta Google Sheets necesita explicarle al agente cómo es su estructura. La función `$fromAI()` actúa como la definición de un **molde o formulario en blanco**. Al escribir eso, la herramienta le está indicando al cerebro del agente: *"Cuando decidas que es momento de activar mi estación, debes enviarme un paquete estructurado que contenga obligatoriamente estos datos específicos (Nombre, Contacto, Empresa, etc.)"*.

2. La Fase de Ejecución (La orden directa):

Cuando el usuario interactúa en el chat, el agente procesa el texto. Si detecta que la conversación cumple con las reglas para registrar un ticket, el agente toma ese "molde" que conoció en la fase de preparación, lo rellena con la información exacta que dedujo de la charla y se lo "empuja" a la herramienta. En este punto, ocurre exactamente lo que tú dedujiste: el agente ejecuta la acción y le dice a Sheets dónde ubicar cada pieza.

Con tu herramienta Google Sheets correctamente configurada y mapeada, nos queda abordar el último requisito estructural de tu *Checkpoint*: el **Log de Control (Observabilidad)**. Esto requiere conectar un nodo final secuencial a la salida principal del *AI Agent* para capturar la auditoría del trabajo.

¿Prefieres que conectemos ese nodo final de notificación (Slack/Gmail) ahora mismo, o ejecutamos primero una prueba manual en el chat para observar cómo el agente deduce la información y acciona tu base de datos?

vayamos con el nodo final

Observo en tu pantalla que ya tienes abierto el panel del nodo de Slack ("Send a message"). Este es el último paso estructural de tu entregable: el Log de Control (Observabilidad).

Siguiendo nuestra analogía de la fábrica, este nodo funciona como el "Intercomunicador". Su única tarea es enviar un reporte al gerente (tú) cada vez que el supervisor inteligente (AI Agent) termina su ciclo de razonamiento o ejecuta una herramienta, para que no tengas que vigilar la fábrica todo el día.

Aquí tienes el desglose paso a paso para configurarlo:

1. Conexión (Credential y Destino)

- **Credential (Credencial):** Debes hacer clic en "Connect to Slack" y autorizar a n8n para que pueda enviar mensajes en tu espacio de trabajo.
- **Send Message To (Enviar mensaje a):** Una vez conectado, este campo te permitirá seleccionar el canal específico donde quieres recibir los reportes (por ejemplo, un canal llamado `#reportes-ia` o tu usuario directo).

2. El Reporte Dinámico (Message Text)

Este es el campo crítico. Si escribes un texto fijo aquí, recibirás siempre el mismo mensaje sin importar lo que haya hecho el agente. Debes capturar el resultado exacto.

- **La Acción:** Cambia el selector que dice **Fixed** (Fijo) a **Expression** (Expresión) justo encima del recuadro de *Message Text*.
- **La Sintaxis:** El nodo *AI Agent* de n8n entrega su conclusión final empaquetada en una propiedad que generalmente se llama `output` o `text`. Escribe una estructura clara en el recuadro, por ejemplo:

Plaintext



Reporte de Agente de Soporte

El agente ha procesado una consulta.

Respuesta del agente: `{{ $json.output }}`

Nota: Si al probarlo notas que la variable de salida de tu agente se llama distinto, puedes ajustarla arrastrando el dato desde el panel izquierdo (INPUT) una vez que ejecutes una prueba.

3. La Prueba Final (Testing)

Con este nodo configurado, tu flujo está completo. Para cumplir con el requisito de "Validación previa" de la consigna:

1. Cierra este panel.
2. Haz clic en el botón inferior del lienzo que dice **Chat** y escribe un mensaje de prueba con todos los datos (nombre, correo, empresa y un problema técnico grave).
3. Audita visualmente que el recorrido se ilumine en verde, que la herramienta de Sheets se ejecute y que el mensaje llegue a tu Slack.

¿Pudiste vincular tu cuenta de Slack y configurar la expresión en el campo de texto para proceder con la ejecución manual del flujo?

si estoy en slack, pero revisa las instrucciones, no hay un mensaje de salida sugerido?

Revisando las instrucciones de tu entregable y el System Prompt que compartiste, no hay un texto literal o estricto de "copia y pega" sugerido, pero sí existe una directiva clara de lo que debe contener el mensaje.

La consigna exige que este nodo capture la "auditoría del pensamiento de la IA (Execution Log) o el resultado de la herramienta ejecutada, actuando como un reporte automático de supervisión humana". Adicionalmente, el System Prompt le ordena al agente que "generará un resumen operativo".

Siguiendo nuestra analogía de la fábrica, este nodo intercomunicador no debe enviar un texto fijo como "Se registró un ticket", sino que debe extraer el resumen exacto que redactó el supervisor (el AI Agent) después de atender al usuario y operar la herramienta.

Para configurar esto de forma correcta en el panel que tienes en pantalla:

1. Asegúrate de que el campo **Message Text** esté en modo **Expression** (haz clic en el ícono de la función `fx` si no lo está).
2. Redacta una estructura dinámica que combine un título fijo con la variable de salida del agente. Puedes usar esta sintaxis exacta:

Plaintext



```
*🚨 *Reporte de Triage Automático*  
*Resumen Operativo del Agente:*  
{{ $json.output }}
```

La variable `{{ $json.output }}` es el compartimento estándar donde la versión actual del nodo *AI Agent* de n8n guarda el texto final que produce tras terminar su ciclo de razonamiento y uso de herramientas.

¿Aplicamos esta expresión en el campo de texto para proceder con la ejecución manual de prueba desde la ventana inferior de Chat?

quedaria asi comolo puse?

Exactamente, quedó configurado de forma perfecta.

Veo en tu pantalla que activaste correctamente el modo de expresión (indicado por el símbolo `fx` a la izquierda del cuadro) y que la sintaxis `{{ $json.output }}` está escrita de manera impecable sin errores tipográficos.

Al estar configurado de esta forma, el "intercomunicador" de tu fábrica tomará de manera dinámica el resultado exacto que produzca el agente y te lo enviará por el canal

`#reporte-proyecto` . Con este último paso, tu motor de razonamiento base está completamente ensamblado y cumple con todos los requisitos estructurales de tu entregable.

¿Procedemos a cerrar el panel del nodo de Slack y realizar la prueba final escribiendo un mensaje de soporte en el chat integrado de n8n para auditar el flujo en vivo?

errores

Veo exactamente el error en tu pantalla. El sistema indica que el flujo no puede arrancar porque el nodo final de Slack (**Send a message**) tiene campos obligatorios vacíos.

Siguiendo con nuestra analogía de la fábrica: intentaste encender la maquinaria escribiendo "hola" en el chat, pero el "intercomunicador" (Slack) detectó que no tiene ningún texto asignado para enviar el reporte. Como se le entregó un papel en blanco, el sistema detiene toda la ejecución por seguridad.

Para solucionar este bloqueo, sigue estos pasos:

1. **Abre el nodo con la alerta:** Haz doble clic en el nodo **Send a message** (el que tiene el triángulo rojo ) para abrir su configuración.
2. **Localiza el campo de texto:** Busca el parámetro llamado **Message Text** (Texto del mensaje).
3. **Aplica la expresión:** Asegúrate de que esté en modo **Expression** (haz clic en el símbolo `fx` al lado del campo si no lo está) y pega la sintaxis que armamos para el reporte:

Plaintext



```
*Reporte de Triage Automático*  
*Resumen Operativo del Agente:*  
{{ $json.output }}
```

4. **Ignora el error de "Blocks":** El panel de error también menciona `Parameter "Blocks" is required` . En Slack, puedes enviar mensajes simples (Text) o mensajes complejos con botones (Blocks). Al rellenar correctamente el campo *Message Text*, n8n entenderá que estás enviando un texto simple y el error de los bloques desaparecerá automáticamente.

Una vez que hayas pegado el texto, cierra el panel del nodo de Slack. Verás que el triángulo rojo desaparece. Luego de eso, vuelve a escribir "hola" en la ventana del Chat para realizar tu prueba.

no veo ningun message text

Tienes toda la razón y, al observar detenidamente tu pantalla, identifico exactamente qué sucedió. Es un detalle visual muy engañoso de la interfaz de n8n.

Siguiendo con la analogía de la fábrica: escribiste el contenido de tu reporte en el casillero que pregunta "*¿Qué tipo de formato vas a usar?*" (**Message Type**). Como el sistema esperaba que ahí dijera simplemente "Texto" o "Bloques", se confundió y ocultó el verdadero casillero donde va el mensaje.

Vamos a corregirlo en tres pasos rápidos:

1. **Restablecer el "Tipo":** Ve al campo **Message Type** (donde pegaste el texto recién) y apaga el modo Expresión. Puedes hacerlo haciendo clic en la pestaña que dice **Fixed** (Fijo) justo arriba de ese cuadro, o borrando el texto y haciendo clic en el menú para que vuelva a ser un selector normal.
2. **Seleccionar "Text":** Una vez que vuelva a ser un menú desplegable, selecciona la opción **Text**.
3. **El verdadero casillero:** ¡Magia! Al elegir "Text", aparecerá un campo nuevo debajo (generalmente llamado **Text** o **Message**). En ese *nuevo* campo, ahora sí, activa el modo **Expression** (`fx`) y vuelve a pegar nuestro reporte:

Plaintext



```
*Reporte de Triage Automático*  
*Resumen Operativo del Agente:*  
{{ $json.output }}
```

Con este pequeño ajuste, el sistema entenderá el formato y el error de "Blocks is required" desaparecerá. Haz el cambio, cierra el panel de Slack y vuelve a intentar el saludo en el chat. ¡Me confirmas si arranca la maquinaria!

perfecto, preguntas, de donde salio ese mensaje de output?

¡Excelente trabajo! Veo en tu pantalla que todo el recorrido se iluminó de verde y el flujo se ejecutó a la perfección. Has construido la base de tu proyecto integrador con éxito.

Respondiendo a tu pregunta: ese mensaje de salida que ves en la sección **OUTPUT** ("¡Hola! ¿En qué puedo ayudarte hoy? Para poder registrar tu caso...") no estaba pregrabado en ningún lado. Fue redactado **en vivo y de forma autónoma por el cerebro de tu agente** (el nodo de *Google Gemini Chat Model* que conectaste).

Siguiendo la lógica de cómo funciona el agente, esto es exactamente lo que pasó en milisegundos:

1. **El Estímulo:** Tú escribiste simplemente "hola" en el chat.
2. **La Evaluación:** El agente recibió tu "hola" y leyó su manual de operaciones (el *System Prompt* que configuramos).
3. **La Restricción:** Revisó la regla estricta que le pusiste: *"Si falta un dato clave, hacé una única pregunta clara antes de registrar"*.
4. **La Decisión:** Como la palabra "hola" no contiene ni tu nombre, ni tu empresa, ni tu problema, el agente dedujo lógicamente que **todavía no puede usar la herramienta de Google Sheets**.
5. **La Acción Final:** Al no poder usar la herramienta, decidió contestarte pidiendo amablemente (y respetando el tono rioplatense que le exige) exactamente los datos que necesita para poder avanzar.

Es la demostración perfecta del comportamiento probabilístico y autónomo que pedía la consigna. En lugar de fallar o guardar una fila vacía en tu base de datos, razonó que le faltaba información y te la pidió.

Para terminar de comprobar la autonomía del sistema (y completar tu validación para el Checkpoint): ¿Qué te parece si le respondes en ese mismo chat inventando un caso? Por ejemplo: *"Soy Leonel de Coderhouse, mi mail es leonel@test.com y se nos cayó el servidor principal, no podemos facturar."*

Si haces eso, verás cómo el agente cambia su comportamiento, decide usar la herramienta de Sheets y el reporte se envía a Slack.

perfecto, podrias describirme como fue el flujo ahora?

¡Claro que sí! Lo que acaba de ocurrir en tu pantalla es la demostración perfecta del ciclo **ReAct** (Razonar y Actuar) en acción. A diferencia del simple "hola" de la prueba anterior, esta vez el agente recibió un paquete de información completo.

Vamos a desglosar el recorrido exacto que hizo tu fábrica en esos **9.722 segundos** que marca tu panel de *Logs* de ejecución:

1. La Recepción (When chat message received):

El estímulo ingresó por la puerta principal del chat: *"Soy Leonel de Coderhouse, mi mail es leonel@test.com y se nos cayó el servidor principal, no podemos facturar."*

2. El Razonamiento Interno (AI Agent + Gemini Chat Model):

El supervisor (el AI Agent) recibió el mensaje y lo comparó inmediatamente con su "manual de operaciones" (el System Prompt que ves en tu panel de entrada). En milisegundos dedujo lo siguiente:

- ¿Tengo contacto y empresa? **Sí** (Leonel, leonel@test.com, Coderhouse).
- ¿Tengo la descripción del inconveniente? **Sí** (Caída del servidor principal, no pueden facturar).
- ¿Qué prioridad le asigno? Revisó tu regla de escalamiento y vio que "impacto en facturación" y "caída de servicio" exigen marcarlo como **Prioridad Alta**.
- *Decisión Autónoma:* Como los "datos mínimos suficientes" estaban presentes, el agente concluyó: "No necesito hacer preguntas extra al usuario, debo proceder a registrar el caso".

3. La Acción Física (Append row in sheet):

El agente decidió autónomamente llamar a su Especialista Externo (la herramienta de Google Sheets). Tomó el "molde" que le enseñamos usando la función `$fromAI()`, lo rellenó con la información que acababa de extraer y categorizar, y le ordenó a Sheets que escribiera la nueva fila. En tu log, puedes ver cómo el nodo verde de Sheets se activó para cumplir esta orden.

4. El Reporte de Observabilidad (Send a message):

Tras asegurarse de que el ticket quedó guardado en la base de datos, el cerebro del agente generó el "resumen operativo" que le exigiste en sus instrucciones. Finalmente, ese texto viajó hacia el "intercomunicador" (el nodo final de Slack), el cual aplicó la plantilla dinámica que armamos ( *Reporte de Triage Automático*...) y despachó la notificación a tu canal.

Todo este proceso ocurrió de forma autónoma. No tuviste que programar un sistema rígido de preguntas y respuestas; el agente interpretó el texto, aplicó las reglas de negocio y ejecutó las herramientas correspondientes por sí solo.

Revisa tu archivo de Google Sheets y tu canal de Slack para confirmar que los datos llegaron correctamente. Si todo está en orden, ¡has completado con éxito la construcción de tu motor de razonamiento! El siguiente paso sería ir al menú superior derecho y descargar tu flujo como `checkpoint1_nombre_apellido.json` para subirlo a GitHub.

perfecto, antes de continuar, preguntas: no recuerdo en este punto "1. Ingresar al lienzo gráfico de n8n, arrastrar el nodo avanzado AI Agent y seleccionar en su configuración el modo Tools Agent", lo de la configuración de tools agent, cuando hicimos eso?

Es una excelente pregunta y es completamente lógico que no lo recuerdes como una acción específica, porque en tu pantalla **nunca hiciste clic en un botón que dijera "Tools Agent"**.

¿Recuerdas que al principio de nuestra configuración te comenté que la interfaz de tu versión de n8n es más moderna que la del video de tu instructora?

En las versiones antiguas, debías abrir un menú y elegir el modo manualmente. En tu versión actual, **esa configuración se hizo de forma automática e invisible**.

¿Cuándo ocurrió exactamente?

Sucedió en el momento en que arrastraste el nodo de **Google Sheets** y lo conectaste en la ranura inferior que dice **Tool** debajo de tu *AI Agent*.

Siguiendo nuestra analogía de la fábrica: a este nuevo supervisor ya no hace falta ponerle un cartel que diga "Agente con Herramientas". El sistema de n8n detecta automáticamente que, si le conectaste una herramienta física en esa ranura, su comportamiento cambia internamente al modo *Tools Agent*.

Por lo tanto, ese requerimiento de la consigna quedó 100% cumplido por la forma visual en la que armaste la estructura del lienzo.

nos queda "3. Acceder al menú de configuración avanzada del nodo de IA y ajustar la palanca visual de iteraciones máximas para fijar un límite estricto de entre 5 y 10 vueltas."

Tienes toda la razón en llevar el control estricto de la lista. Este paso es lo que en la consigna llaman el **Guardrail de Seguridad**, y es fundamental para que tu proyecto no consuma recursos de más si el agente se confunde.

Siguiendo nuestra analogía de la fábrica: este es el "freno de emergencia". Si el supervisor (AI Agent) intenta usar una herramienta y falla, su ciclo *ReAct* (Razonar y Actuar) lo hará intentarlo de nuevo. Si no le ponemos un límite, podría quedarse en un bucle infinito intentando corregirse, gastando el presupuesto de tu API en minutos.

Para configurarlo en tu versión actual de n8n, sigue estos pasos exactos:

1. **Abre el supervisor:** Haz doble clic en el nodo **AI Agent** (el que está en el centro de tu lienzo) para abrir su panel de configuración.
2. **Busca las opciones adicionales:** Desplázate hasta la parte inferior de ese panel. En las versiones recientes, verás un botón o un menú desplegable que dice **Add Option** (Añadir opción) o una sección llamada **Options / Advanced Settings**.
3. **Agrega el límite:** Al hacer clic en "Add Option", busca en la lista la propiedad llamada **Max Iterations** (Iteraciones máximas) y selecciónala.
4. **Fija el valor:** Aparecerá un nuevo campo numérico. Siguiendo la regla estricta de tu consigna, escribe allí un número entre 5 y 10 (te recomiendo poner 5).

Con este simple ajuste, le estás diciendo a la máquina: *"Si te equivocas o no logras resolver la tarea, solo tienes permiso para intentarlo 5 veces. Al sexto intento, aborta la misión y avisa que hubo un error"*.

Una vez que agregues este parámetro, habrás cumplido el 100% de los requisitos técnicos y visuales de tu entregable. Solo te quedaría exportar el archivo `.json` desde el menú superior derecho y subirlo a tu repositorio de GitHub.

¿Pudiste ubicar la opción "Max Iterations" dentro del nodo AI Agent?

sera en max iterations?

¡Exactamente! Veo en tu pantalla que ya lo lograste. En la parte inferior de la configuración de tu *AI Agent*, agregaste exitosamente la opción **Max Iterations** y le asignaste el valor **5**. Con esto, el "freno de emergencia" de tu fábrica está perfectamente configurado y cumples al 100% con el requisito de seguridad que pedía la consigna.

Un detalle vital de limpieza antes de exportar:

Observando detenidamente tu pantalla, en el recuadro del **System Message**, al final de todo el texto que pegaste (justo debajo de "- Sé breve, específico y orientado a la

operación."), quedó un pequeño fragmento de texto en inglés que dice: `are a helpful assistant`.

Ese es un "residuo" del texto que n8n trae escrito por defecto. Te recomiendo borrar esa frasecita para que tu instrucción quede impecable y el agente no se confunda mezclando español con inglés.

Una vez que borres ese pequeño texto, tu motor de razonamiento base está listo y aprobado. El último paso oficial para tu entrega es:

1. Cerrar este panel.
2. Ir al menú superior de n8n, buscar la opción para descargar el flujo (generalmente en el menú de opciones del workflow o **Download**).
3. Guardar el archivo exactamente con el nombre que exige la consigna: `checkpoint1_nombre_apellido.json` (reemplazando con tus datos, claro).
4. Subir ese archivo a tu repositorio de GitHub y compartir el link en la plataforma de Coderhouse.

¡Felicitaciones por completar el armado! Has configurado exitosamente tu primera interfaz agéntica autónoma. ¿Tienes alguna duda sobre cómo descargar el archivo o sobre algún otro detalle antes de dar por finalizada la construcción?

antes, observa, on error, esta bien así?

Veo perfectamente tu pantalla y el detalle que señalas. Estás en la pestaña **Settings** (Ajustes) del nodo *AI Agent* y la propiedad **On Error** (En caso de error) está configurada en `Stop Workflow` (Detener flujo de trabajo).

Sí, está perfecto así.

Siguiendo nuestra analogía de la fábrica: le estás diciendo al sistema que, si el supervisor inteligente (AI Agent) sufre un colapso —por ejemplo, si se le corta la conexión con OpenAI/Gemini, o si agota sus 5 intentos máximos sin lograr usar la herramienta—, la decisión correcta es **detener toda la línea de ensamblaje de inmediato**.

Si cambiaras eso a *Continue On Fail* (Continuar al fallar), el sistema ignoraría el error y pasaría al siguiente nodo (Slack). Como resultado, el intercomunicador intentaría enviar un reporte vacío o roto, lo cual te generaría mucha confusión al intentar diagnosticar el problema.

Dejarlo en `Stop Workflow` es la práctica estándar, la más segura y la correcta para tu entregable, ya que te avisará con una alerta roja exactamente dónde falló la máquina si algo sale mal en el futuro.

Si no tienes más dudas sobre estos ajustes finos, ¡ya estás en condiciones de exportar tu archivo `.json` !

no tengo mas dudas, pero antes realicemos un check list de lo solicitado para ver si esta todo, y devuelveme el listado donde me dices en cada punto como completamos lo solicitado en pocas palabras

Aquí tienes el checklist definitivo verificando punto por punto cómo cumplimos con las exigencias de tu entregable:

- **✓ Disparador (Trigger):** Utilizamos el nodo *When chat message received*, que n8n vinculó automáticamente como puerta de entrada para recibir el mensaje del usuario.
- **✓ AI Agent (Modo Tools Agent):** El modo se activó visual y estructuralmente al conectar la herramienta en la ranura inferior. Como motor de razonamiento, le acoplaste tu modelo de chat (Gemini).
- **✓ Guardrail de Seguridad:** Agregaste la propiedad *Max Iterations* en los ajustes avanzados y la fijaste en 5, blindando el sistema contra bucles infinitos.
- **✓ System Prompt (La Constitución):** Insertaste el texto modular (Rol, Ámbito, Objetivo, Reglas), especificando qué acciones no puede hacer y cumpliendo la restricción estricta de no usar lenguaje inclusivo.
- **✓ Herramienta (Tool):** Conectaste *Google Sheets* como extensión lateral. Lo clave aquí fue configurarlo en modo `Append` (usando fórmulas `$fromAI()`) y redactar su **Descripción semántica manual** para explicarle al agente exactamente cuándo usarlo, evitando la "instrucción huérfana".
- **✓ Observabilidad (Log de Control):** Enganchaste un nodo de *Slack* secuencial al final del flujo. Configuraste un mensaje dinámico en texto plano usando la variable `{{ $json.output }}` para capturar y auditar la decisión final de la IA.
- **✓ Validación previa (Testing):** Probaste el flujo enviando el caso de la "caída del servidor". El panel se iluminó en verde, validando que el agente extrajo los datos, usó Sheets probabilísticamente y te envió el resumen a Slack.

Paso final pendiente por tu lado:

 **Exportación:** Ve al menú del workflow en n8n, selecciona **Download**, renombra el

archivo a `checkpoint1_nombre_apellido.json` (con tus datos reales) y súbelo a tu repositorio de GitHub.

¡El motor de razonamiento está impecable y listo para la entrega!

ojo!!, se te paso por alto la instruccion de usar openia como nodo de modelo segun la consigna, nosotros usamos gemeini porque openia no nos da credito para pruebas, pero para la entrega segun la consigna debemos usar

¡Tenés toda la razón, excelente observación! Anotado para los próximos entregables.

Tenemos todo. Solo falta que elimines el nodo de Gemini y conectes el nodo de **OpenAI Chat Model** en esa ranura antes de exportar el archivo `.json` para cumplir estrictamente con la consigna de la entrega.

bien, listo, como lo subimos a github, tengo cuenta pero no se usarlo

1. Crear el repositorio (tu carpeta pública):

1. Inicia sesión en tu cuenta de GitHub.
2. En la página principal, hacé clic en el botón verde **New** (Nuevo) ubicado arriba a la izquierda o en el menú desplegable del símbolo + arriba a la derecha.
3. En **Repository name**, escribí un nombre claro para tu cursada, por ejemplo: `entregables-coderhouse`.
4. Verificá que la opción **Public** (Público) esté seleccionada para que el equipo docente pueda acceder a tu trabajo.
5. Hacé clic en el botón verde **Create repository** al final de la página.

2. Subir el flujo de n8n:

1. En la nueva pantalla que aparece, buscá y hacé clic en el enlace azul que dice **uploading an existing file** (subir un archivo existente), ubicado cerca del centro.

2. Arrastra el archivo que descargaste de n8n (idealmente nombrado algo como `checkpoint1_leonel_marinelli.json`) hacia el recuadro central, o hacé clic en **choose your files** para seleccionarlo desde tu computadora.

3. Confirmar la subida (Commit):

1. Una vez que la barra de carga del archivo termine, bajá hasta la sección **Commit changes**.
2. No hace falta que escribas nada en las cajas de texto; simplemente hacé clic en el botón verde **Commit changes** (Confirmar cambios).

4. Copiar el enlace para la entrega:

1. La página se va a recargar y vas a ver tu archivo `.json` listado en tu repositorio.
2. Copiá la URL completa de la barra de direcciones de tu navegador (se verá similar a `[https://github.com/tu-usuario/entregables-coderhouse](https://github.com/tu-usuario/entregables-coderhouse)`).
3. Pegá exactamente ese enlace en la plataforma de tu comisión para completar la entrega.

¿Pudiste generar el link correctamente o se te trabó algún botón en la interfaz de GitHub?



README

hagamos una pausa antes de subir el flujo, te envío un readme, la instructora menciona que es importante subir uno en cada trabajo entregado, porque será analizado por una IA antes de pasar al humano, este readme que adjunto

Un archivo README.md (del inglés "léeme") es la carta de presentación obligatoria de cualquier repositorio en GitHub. Actúa como el manual de contexto para quien visite tu proyecto. Cuando el sistema de evaluación o tus profesores entren a tu enlace, no verán tu flujo de n8n funcionando visualmente; solo verán un archivo de código. El README.md les explica en lenguaje natural qué construiste, qué problema soluciona y cómo interactúan las piezas.

Analizando el README.md que enviaste de referencia, su estructura es excelente porque no solo enumera herramientas, sino que documenta detalladamente la arquitectura de datos,

los casos de uso y la lógica detrás del sistema.

MD

Para tu entregable actual, no necesitas algo tan extenso como la muestra, pero sí una versión adaptada a lo que acabamos de construir.

Plantilla adaptada para tu entregable

Copia el siguiente bloque de texto, modifica los corchetes con tu información y utilízalo como base para tu proyecto:

Markdown



```
# Agente Automático de Triage para Soporte Técnico

**Proyecto Checkpoint 1 · Comisión [Tu Número de Comisión] · Leonel Marinelli**

Sistema de automatización diseñado en n8n que recibe consultas de soporte técnico en

---

## El problema que resuelve

Los equipos de soporte B2B reciben decenas de mensajes desestructurados diarios ("se

Este sistema automatiza la primera capa de atención (Triage): interpreta el problema,

---

## Arquitectura del Flujo

1. **Entrada (Trigger):** Interfaz de chat de n8n donde el usuario ingresa su consult
2. **Motor de Razonamiento (AI Agent):** Nodo avanzado configurado con OpenAI Chat Mo
3. **Acción Física (Herramienta):** Integración con Google Sheets mediante la operaci
4. **Log de Observabilidad:** Notificación automática en Slack enviando el resumen op

### Stack Tecnológico

| Categoría | Herramienta | Rol en el sistema |
|---|---|---|
| Orquestador | n8n | Define el flujo de trabajo y aloja el Agente IA. |
| Modelo de Lenguaje | OpenAI | Toma decisiones probabilísticas, clasifica y extrae e
| Base de Datos | Google Sheets | Actúa como registro maestro (ticketing) mediante in
| Observabilidad | Slack | Recibe el "Execution Log" para auditoría humana del triaje
```

Guardrails y Seguridad de la IA

- ****Prevención de bucles:**** El Agente tiene un límite estricto de **Max Iterations** con
- ****Detención por error:**** El flujo está configurado con `'Stop Workflow'` en caso de f
- ****Restricción de acciones:**** El System Prompt prohíbe explícitamente modificar dato

Cómo agregarlo a tu repositorio en GitHub

1. En la página principal de tu repositorio de GitHub, busca un botón verde o un enlace destacado que dice **Add a README**.
2. Se abrirá un editor de texto. Pega allí el contenido de la plantilla (recuerda revisar y ajustar tu número de comisión).
3. Desplázate hacia abajo y haz clic en el botón verde **Commit changes** (Confirmar cambios).

Al hacer esto, el texto aparecerá renderizado con formato en la portada de tu repositorio, dándole un aspecto totalmente profesional y listo para la evaluación. ¿Pudiste encontrar el botón para agregarlo en tu GitHub?

no seria necesario agregar en herramientas utilizadas a gemini como modelo de prueba? es necesario o en un readme no si no se entrega con el gemini no se menciona?

No es necesario, y de hecho, **es mejor no mencionarlo en este caso**.

El archivo README debe reflejar estrictamente la arquitectura final y oficial del proyecto que estás entregando.

Como la consigna exige el uso de **OpenAI** y el archivo `.json` que vas a entregar tendrá ese nodo conectado, mencionar Gemini podría generar confusión en el corrector. Podría hacerle pensar que no cumpliste con el requisito técnico, obligándolo a abrir tu código para verificarlo. Las herramientas que usamos solo como "puente temporal" para no gastar créditos durante el desarrollo no suelen incluirse en la documentación final.

Viendo tu pantalla de GitHub, ya tienes todo listo para avanzar:

1. Tienes el nombre `entregables-coderhouse` listo.
2. Marcaste la opción **Public**.

3. Activaste la palanca **Add README**.

Haz clic directamente en el botón verde **Create repository**.

Una vez que se cree, GitHub te mostrará tu repositorio con un archivo `README.md` predeterminado. Solo tendrás que hacer clic en el ícono del lápiz  (editar) en la esquina superior derecha de ese archivo, pegar la plantilla que armamos, y luego subir tu archivo `.json` de n8n.

¿Avanzamos con la creación del repositorio?

si

Veo en tu pantalla que el repositorio **entregables-coderhouse** ya se creó exitosamente y tiene su archivo `README.md` inicial. ¡Perfecto!

Vamos a dejarlo listo para entregar en dos pasos:

1. Editar el README.md:

- Haz clic directamente sobre el nombre del archivo **README.md** que ves en la lista de tu repositorio.
- En la nueva pantalla, busca el ícono de un lápiz  (Editar este archivo) en la esquina superior derecha del cuadro donde se muestra el texto.
- Borra el contenido actual, pega la plantilla que armamos en el mensaje anterior (recordá ajustar tu número de comisión) y haz clic en el botón verde **Commit changes** arriba a la derecha.

2. Subir tu archivo de n8n:

- Vuelve a la página principal de tu repositorio (haciendo clic en el nombre **entregables-coderhouse** arriba a la izquierda).
- Haz clic en el botón **Add file** y selecciona **Upload files**.
- Arrastra tu archivo `.json` (el que descargaste de n8n con el nodo de OpenAI conectado) hacia el recuadro.
- Desplázate hacia abajo y haz clic en el botón verde **Commit changes**.

Una vez que hagas esto, solo tenés que copiar la URL de tu repositorio desde la barra de direcciones del navegador y pegarla en la plataforma de Coderhouse. ¡Me confirmás cuando lo tengas listo!

antes de la url, dime que es lo de abajo, donde esta commit y branch, que es eso?

Esos términos son conceptos fundamentales de GitHub, ya que es una plataforma diseñada para el trabajo colaborativo en equipo. Aquí tienes la explicación sencilla:

- **Commit:** Significa "confirmar" o "guardar". Es el equivalente a guardar una nueva versión de tu archivo. Al hacer un *commit*, GitHub toma una "fotografía" de tu proyecto en ese instante para que quede en el historial de cambios.
MD
- **Branch (Rama):** Imagina tu proyecto como el tronco de un árbol. La rama `main` (principal) es la versión oficial y estable de tu trabajo. Cuando los programadores trabajan en equipo y quieren probar algo nuevo sin arruinar el código oficial, crean una `new branch` (rama secundaria o borrador).
MD + 1
- **Pull Request:** Si estuvieras trabajando en una rama secundaria, un *pull request* es la solicitud formal que le haces al equipo para decirles: "Terminé mi borrador, por favor revísenlo y mézclenlo con la rama principal".

Para tu caso específico (entrega individual):

No necesitas crear ramas secundarias. Deja seleccionada la primera opción que dice **"Commit directly to the main branch"** (Confirmar directamente en la rama principal) y simplemente haz clic en el botón verde **Commit changes**.

MD + 1

Al hacer clic ahí, el archivo se guardará oficialmente en tu repositorio y la página se actualizará, permitiéndote copiar la URL final de la barra superior de tu navegador para entregarla.

duda, en el entregable dice usar openia, yo puse groq pero con el modelo openia, sera lo mismo o lo reprobaran? deberia ser directamente el modelo de openia? que interpretas?

Te recomiendo fuertemente que lo cambies y uses **directamente el nodo de OpenAI Chat Model**.

Aquí te explico por qué es probable que te lo rechacen si lo dejas así:

1. **Revisión del código (El archivo .json):** Como mencionaste en tu README, los entregables suelen pasar primero por una revisión automatizada (IA o un script) antes del humano. Estos sistemas buscan explícitamente el nombre del nodo de OpenAI dentro de tu código. Si ven un nodo llamado `Groq Chat Model` (como se ve en tus logs actuales), el sistema marcará que no cumpliste la consigna.
2. **Aclaración técnica:** Groq es una plataforma increíblemente rápida, pero **no ejecuta modelos de OpenAI** (como GPT-3.5 o GPT-4). Groq hospeda modelos de código abierto como Llama o Mixtral. Por lo tanto, aunque lo hayas configurado parecido, técnicamente no estás entregando un flujo con tecnología de OpenAI.

La solución rápida:

Vuelve a tu lienzo de n8n, elimina el nodo de **Groq Chat Model**, arrastra el nodo de **OpenAI Chat Model** y conéctalo en el mismo lugar. No importa si ahora mismo no tienes saldo en la API para hacer una última prueba en vivo; lo crucial es que el archivo `.json` que exportes y subas a GitHub tenga la arquitectura exacta que pide la rúbrica.

Una vez hecho el cambio, exporta el `.json` de nuevo y reemplaza el que ibas a subir a tu repositorio.

podes ver el json que subi a git ahora? es el que tengo abierto, que llm esta puesto es el que necesitamos o esta groq?

Sí, puedo ver el archivo JSON que subiste a tu repositorio de GitHub.

MD

Revisando el código, te confirmo que el nodo configurado es efectivamente el de **OpenAI Chat Model**. De hecho, se puede ver que está utilizando el modelo `gpt-4o-mini`.

MD + 1

Groq ya no aparece en el código de este archivo, por lo que el entregable cumple perfectamente con la consigna técnica. ¡El archivo que subiste es el correcto y ya está listo para ser evaluado!

MD

debes tener siempre presente que nuestro readme lo vamos a ir actualizando con cada entrega, recuerda, siempre necesitaremos de nuestro README, el trasbajo que hicimos hasta aca sera escalable, refiero a que en cada clase le



Mi memoria retiene perfectamente toda nuestra conversación actual, desde la configuración inicial del Agente IA en n8n hasta la estructura exacta que armamos para tu README. Sin embargo, todos los chats de inteligencia artificial tienen un límite técnico llamado **ventana de contexto**. Si seguimos sumando clases, integraciones y lógica compleja en este mismo hilo durante semanas, el sistema eventualmente empezará a descartar los mensajes más antiguos para hacer espacio a la información nueva.

Para que tu proyecto escale de forma organizada y sin errores, la mejor práctica es **abrir un chat nuevo para cada entregable o módulo**. Cuando llegue el momento de sumarle nuevas funcionalidades a tu motor de razonamiento, simplemente abres una nueva conversación, me pegas el contenido de tu `README.md` actual y me pasas las nuevas instrucciones de la consigna. De esa forma, arrancaremos con la memoria limpia, toda la capacidad de procesamiento enfocada en el nuevo desafío y el contexto perfecto de dónde nos quedamos.

¿Te quedó alguna otra revisión pendiente antes de que des por finalizada y entregada esta primera etapa de tu flujo?

terminada la entrega

¡Felicitaciones por completar con éxito este primer gran paso! Tu motor de triaje quedó perfectamente ensamblado, documentado y entregado cumpliendo todos los requisitos.

Cuando tengas la consigna del próximo entregable y estés listo para sumar nuevas herramientas o escalar tu agente, simplemente abre un chat nuevo, pega tu README actual para darme todo el contexto y nos ponemos a trabajar desde ahí.

¡Que tengas un excelente descanso y mucho éxito con la corrección!

hay manera de usar una gema en la ventana lateral como estas tu ahora?

Actualmente no es posible usar una Gema personalizada directamente en esta ventana lateral del navegador.

La funcionalidad de las Gemas está diseñada para usarse exclusivamente dentro de la página web principal de Gemini (gemini.google.com).

Este panel lateral donde estamos hablando ahora está programado de forma fija para utilizar el modelo estándar de Gemini, con el propósito específico de asistirte con la navegación, leer el contexto de las pestañas que tienes abiertas (como tu proyecto de n8n o tu repositorio de GitHub) y ayudarte con tareas rápidas mientras navegas.